

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP

ĐỀ TÀI

**XÂY DỰNG HỆ THỐNG TƯ VẤN NỘI QUY
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI**

Giảng viên hướng dẫn : TS. Lương Thái Lê
Sinh viên thực hiện : Lỗ Nguyễn Thành Vinh
Mã sinh viên : 201200415
Lớp : CNTT5
Khóa : 61

Hà Nội – 2024

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

ĐỀ TÀI

**XÂY DỰNG HỆ THỐNG TƯ VẤN NỘI QUY
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI**

Giảng viên hướng dẫn : TS. Lương Thái Lê
Sinh viên thực hiện : Lỗ Nguyễn Thành Vinh
Mã sinh viên : 201200415
Lớp : CNTT5
Khóa : 61

Hà Nội – 2024

LỜI CẢM ƠN

Đầu tiên, em xin được gửi lời cảm ơn chân thành đến TS. Lương Thái Lê – giảng viên hướng dẫn đồ án tốt nghiệp của em. Trong quá trình thực hiện và hoàn thiện đồ án, em đã được nhận sự hỗ trợ và giúp đỡ vô cùng lớn đến từ cô.

Em cũng xin gửi lời cảm ơn đến các thầy cô giáo của khoa Công nghệ thông tin – Đại học Giao thông vận tải đã nhiệt tình giảng dạy, truyền đạt cho em các kiến thức và kỹ năng cần thiết trong suốt quá trình học tập tại trường để có thể thực hiện và hoàn thành đồ án tốt nghiệp này.

Em cũng xin gửi lời cảm ơn đến gia đình, các anh chị tiền bối và bạn bè đã chỉ dẫn, ủng hộ và đồng hành cùng em trong chặng đường theo học tại trường.

Trong đồ án tốt nghiệp lần này, với kiến thức, kỹ năng và kinh nghiệm còn non trẻ của bản thân, em khó tránh khỏi những sai sót, thiếu sót không đáng có. Em rất mong muốn được nhận sự chỉ bảo và góp ý chân thành đến từ các thầy cô giáo, để đồ án của em được hoàn thiện hơn và đồng thời nâng cao kiến thức, kỹ năng của bản thân để có thể áp dụng trong cuộc sống.

Hà Nội, Ngày ... tháng ... năm 2024

Sinh viên thực hiện

Vinh

Lỗ Nguyễn Thành Vinh

MỤC LỤC

Lời cảm ơn.....	3
Mục lục	4
Danh mục các từ viết tắt.....	6
Danh mục bảng biểu.....	7
Danh mục hình ảnh.....	8
Mở đầu.....	10
Chương 1. Tổng quan bài toán xây dựng hệ thống tư vấn nội quy trường đại học giao thông vận tải	12
1.1 Đặt vấn đề	12
1.2 Mục tiêu và phạm vi đề án.....	13
1.3 Công nghệ và công cụ sử dụng	13
1.3.1 Công nghệ sử dụng.....	13
1.3.2 Công cụ sử dụng.....	14
Chương 2. Mô hình ngôn ngữ lớn.....	15
1 Tổng quan mô hình ngôn ngữ lớn	15
2 Kiến trúc Transformer	16
2.1 Tổng quan kiến trúc.....	16
2.2 Encoder – Decoder	17
2.3 Cơ chế self-attention	18
3 Prompt Engineering	19
Chương 3. Truy vấn tạo sinh tăng cường.....	21
3.1 Tổng quan về RAG	21
3.2 Phân loại RAG	22
3.3 Quy trình thực hiện phương pháp RAG ngây thơ (Naive RAG).....	23
3.2.1 Indexing.....	24
3.2.2 Nhúng từ (Embedding).....	26
3.2.3 Retrieving.....	27
3.2.4 Generating.....	28
Chương 4. Xây dựng hệ thống tư vấn nội quy trường Đại học Giao thông vận tải.....	30
4.1 Phân tích thiết kế hệ thống.....	30

4.1.1 Sơ đồ ca sử dụng	30
4.1.2 Đặc tả các chức năng.....	31
4.2 Xây dựng trang web tư vấn viên nội quy	35
4.2.1 Cài đặt chatbot sử dụng phương pháp RAG.....	35
4.2.2 Phân lớp câu hỏi của người dùng	39
4.2.3 Giao diện trang web tư vấn viên nội quy	40
4.3 Xây dựng trang web quản trị tư vấn nội quy.....	41
4.3.1 Giao diện chức năng xác thực	41
4.3.2 Giao diện chức năng thống kê.....	43
4.3.3 Giao diện chức năng cập nhật dữ liệu	44
KẾT LUẬN VÀ KIẾN NGHỊ.....	47
DANH MỤC TÀI LIỆU THAM KHẢO	49

DANH MỤC CÁC TỪ VIẾT TẮT

Dạng viết tắt	Dạng đầy đủ	Ý nghĩa
LLM	Large Language Model	Mô hình ngôn ngữ lớn
RAG	Retrieval Augmented-Generation	Truy vấn tạo sinh tăng cường
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
CSDL	Cơ sở dữ liệu	

DANH MỤC BẢNG BIỂU

Bảng 2.1: Bảng thống kê số lượng tham số của một số LLM phổ biến. 1M = 1,000,000 tham số; 1B = 1,000,000,000 tham số; 1T = 10B	15
Bảng 3.1: Bảng chi phí tiêu tốn khi sử dụng một số mô hình của OpenAI. Nguồn: https://openai.com/api/pricing/	24
Bảng 3.2: Số lượng token tối đa có thể nhúng và số chiều của vector nhúng đầu ra của một số mô hình embedding phổ biến	27
Bảng 4.1: Mô tả các chức năng có trong hệ thống tư vấn nội quy trường Đại học Giao thông vận tải	30
Bảng 4.2: Đặc tả chức năng UC01 - Hỏi đáp	31
Bảng 4.3: Đặc tả chức năng UC02 - Xác thực	32
Bảng 4.4: Đặc tả chức năng UC03 - Thống kê.....	33
Bảng 4.5: Đặc tả chức năng UC04 - Cập nhật dữ liệu	34

DANH MỤC HÌNH ẢNH

Hình 2.1 Sơ đồ các lớp neural trong kiến trúc mô hình Transformer. Nguồn: Vaswani, Ashish, et al. "Attention is all you need." Advances in neural information processing systems 30 (2017).....	17
Hình 3.1: So sánh giữa ba dạng RAG chính. Nguồn: Gao, Yunfan et al. "Retrieval-Augmented Generation for Large Language Models: A Survey." ArXiv abs/2312.10997 (2023) [12].....	22
Hình 3.2: Quy trình thực hiện kỹ thuật RAG Naive.....	24
Hình 4.1: Sơ đồ ca sử dụng của hệ thống tư vấn nội quy sinh viên trường Đại học Giao thông vận tải.	30
Hình 4.2: Ví dụ về một đối tượng Document được đọc bằng PyPDF từ sổ tay sinh viên .	35
Hình 4.3: Giao diện trang quản trị CSDL vector trên Pinecone.....	36
Hình 4.4: Kiểm thử quá trình trả về tài liệu truy vấn từ Pinecone với câu hỏi truy vấn là "Điều kiện được nhận đồ án tốt nghiệp", số tài liệu trả về là 10. Chỉ nội dung của văn bản liên quan được in ra, mỗi văn bản cách nhau bởi một dòng.....	37
Hình 4.5: Kiểm thử mô hình khi chạy với phương pháp RAG trên sổ tay sinh viên với câu hỏi "số điện thoại liên hệ".....	39
Hình 4.6: Sơ đồ cấu trúc hệ thống tư vấn sinh viên đã bổ sung thêm mô hình phân lớp...	40
Hình 4.7: Giao diện trang web tư vấn viên nội quy	40
Hình 4.8: Giao diện trang web tư vấn viên nội quy khi sinh viên đặt câu hỏi	41
Hình 4.9: Giao diện trang đăng nhập vào tài khoản quản trị của trang web	41
Hình 4.10: Giao diện trang đăng nhập báo lỗi khi người dùng nhập không đúng tài khoản	42
Hình 4.11: Giao diện chức năng đổi mật khẩu của tài khoản quản trị	42
Hình 4.12: Giao diện chức năng tra cứu biến động chi phí của tài khoản OpenAI	43
Hình 4.13: Giao diện chức năng thống kê của trang web	43
Hình 4.14: Giao diện của chức năng cập nhật thông tin về CSDL vector store.....	44
Hình 4.15: Giao diện xem thông tin của index.....	44
Hình 4.16: Giao diện thay đổi chủ đề cho index	45

Hình 4.17: Giao diện xác nhận xóa index	45
Hình 4.18: Giao diện truy vấn index	46
Hình 4.19: Giao diện cập nhật bổ sung dữ liệu	46

MỞ ĐẦU

Trường Đại học Giao thông vận tải là một trường đại học lớn của Việt Nam. Hiện nay, trường có hàng chục nghìn sinh viên theo học và mỗi năm lại tăng lên. Đây là một áp lực rất lớn đến bộ máy quản lý sinh viên của nhà trường. Số lượng sinh viên tăng, các vấn đề của sinh viên gặp phải cũng sẽ tăng lên, đòi hỏi bộ phận hỗ trợ sinh viên của nhà trường phải làm việc rất vất vả. Thậm chí, chất lượng của việc hỗ trợ sinh viên có thể không đảm bảo. Sinh viên với tâm lý muốn giải quyết ngay khó khăn khúc mắc của bản thân, tuy nhiên do nhiều vấn đề, chẳng hạn như ngại hỏi, không biết tìm kiếm thông tin ở đâu, hoặc do bộ phận hỗ trợ sinh viên chưa giải quyết kịp thời, sẽ gây ảnh hưởng đến tâm lý sinh viên và có thể là quá trình theo học tại trường. Do đó, việc giải quyết nhanh chóng các vấn đề của sinh viên, đặc biệt là vấn đề nội quy, vốn là vấn đề thường xuyên bị hỏi, trở nên cần thiết.

Trên thực tế, bất kể sinh viên nào cũng đều được cấp một quyển sổ tay sinh viên. Bên trong sổ tay sinh viên chứa toàn bộ các thông tin về nội quy trường. Ngoài ra, các tài liệu văn bản khác được nhà trường cập nhật ở rất nhiều nơi, chẳng hạn như trang qltd.utc.edu.vn, trang utc.edu.vn, hoặc các tài liệu dạng PDF. Dù vậy, với tâm lý ngại tra cứu, sinh viên thường hay hỏi hơn là trực tiếp đi tìm. Bài toán đặt ra là cần một hệ thống có thể giúp sinh viên tra cứu các thông tin liên quan đến câu hỏi của sinh viên, và phản hồi lại một cách tự nhiên giống như con người.

Hiện nay, mô hình ngôn ngữ lớn (LLM) đang ngày càng phát triển. LLM hiện nay có thể sinh câu trả lời bằng ngôn ngữ tự nhiên rất giống con người, ngoài ra còn có khả năng hiểu ngôn ngữ và tóm tắt ngôn ngữ. Hạn chế của LLM đó là nguồn tri thức của LLM bị giới hạn bởi tri thức tham số, do đó nó không thể cập nhật được các tri thức mới nhất. Phương pháp truy vấn tạo sinh tăng cường (RAG) cũng đã được giới thiệu, cho phép cải thiện độ chính xác và tính kịp thời của LLM, giúp LLM có thể trả lời các câu hỏi mang tính thời điểm mà không cần phải huấn luyện lại. Từ vấn đề gặp phải và phương hướng giải quyết trên, tôi quyết định sử dụng LLM và phương pháp RAG làm phương án chủ đạo để xây dựng hệ thống tư vấn nội quy trường đại học Giao thông vận tải trong đồ án tốt nghiệp này.

Quá trình thực hiện đồ án tốt nghiệp này được tổng hợp thành báo cáo gồm 4 chương chính, đó là:

Chương 1 – Tổng quan bài toán xây dựng hệ thống tư vấn nội quy trường đại học Giao thông vận tải: Chương này giới thiệu về bài toán xây dựng hệ thống tư vấn nội quy trường đại học Giao thông vận tải, mục tiêu của đề tài, các công cụ và công nghệ được sử dụng.

Chương 2 – Mô hình ngôn ngữ lớn: Chương này tôi sẽ trình bày về tổng quan mô hình ngôn ngữ lớn, là xương sống của hệ thống tư vấn này. Ngoài ra, tôi sẽ trình bày về kiến trúc transformer, là kiến trúc nền tảng của bất kỳ một LLM nào, và kỹ thuật prompt engineering giúp tối ưu hóa truy vấn đến LLM.

Chương 3 – Truy vấn tạo sinh tăng cường: Chương này tôi sẽ trình bày về phương pháp truy vấn tạo sinh tăng cường, phân loại các phương pháp RAG và quy trình thực hiện phương pháp RAG ngây thơ, là phương pháp chủ đạo được tôi sử dụng để xây dựng hệ thống.

Chương 4 – Xây dựng hệ thống tư vấn nội quy trường đại học Giao thông vận tải: Trong chương này, tôi sẽ trình bày về việc phân tích hệ thống, từng bước chi tiết xây dựng hệ thống và giao diện của sản phẩm cuối cùng.

Thông qua đồ án tốt nghiệp lần này, tôi mong muốn có thể xây dựng một hệ thống tư vấn nội quy cho sinh viên trường đại học Giao thông vận tải với cấu trúc đơn giản và chính xác nhất có thể, nhằm hỗ trợ chính các bạn sinh viên và thầy cô bộ phận hỗ trợ của trường. Trong quá trình làm đồ án, tôi khó tránh khỏi những sai sót và thiếu sót về kiến thức, hình thức trình bày hoặc nội dung. Tôi hi vọng được các thầy cô, các anh chị và các bạn đọc có thể đóng góp ý kiến, để đồ án tốt nghiệp của tôi được hoàn thiện hơn.

CHƯƠNG 1. TỔNG QUAN BÀI TOÁN XÂY DỰNG HỆ THỐNG TƯ VẤN NỘI QUY TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI

1.1 Đặt vấn đề

Trường Đại học Giao thông Vận tải là một trường đại học lớn tại Việt Nam, với số lượng sinh viên theo học tại trường là vô cùng lớn và tăng dần theo từng năm. Điều này tạo một áp lực vô cùng lớn đến các phòng ban phụ trách việc quản lý các vấn đề của sinh viên. Số lượng sinh viên lớn, đồng nghĩa số lượng vấn đề xảy ra đối với sinh viên cũng rất lớn. Việc giải đáp ỏn thỏa các thắc mắc của sinh viên là điều cần thiết, mục đích nhằm giúp sinh viên có thể học tập và hoạt động hiệu quả tại trường. Ngoài ra, nắm bắt được những vấn đề của sinh viên còn giúp nhà trường nắm bắt được những vấn đề phát sinh trong trường, từ đó điều chỉnh để giúp quản lý sinh viên hiệu quả hơn.

Nhà trường hiện cũng đã có các phòng ban hỗ trợ sinh viên. Ngoài ra, ở các khoa, s các cán bộ phụ trách văn phòng khoa cũng hoàn toàn có thể giúp đỡ để giải đáp các thắc mắc của sinh viên. Tuy nhiên, số lượng các thầy cô phụ trách văn phòng khoa lại có giới hạn. Do đó, gánh nặng công việc của các thầy cô thực sự lớn, có thể gây tâm lý mệt mỏi, không thoải mái khi tiếp xúc với sinh viên. Ngoài ra, các câu hỏi của sinh viên đặt ra thường đã có trong các tài liệu của trường, tuy nhiên, sinh viên thường có tâm lý ngại tra cứu, ngại liên hệ với các thầy cô, không nắm bắt được các thông tin của các thầy cô hỗ trợ, đặc biệt thiếu hiểu biết về các nội quy quy định đã có của trường, đặc biệt là sinh viên mới nhập học, khó nắm bắt được hết các tài liệu đã có ở trường. Thậm chí, có thể có một vấn đề được rất nhiều sinh viên quan tâm, các thầy cô phụ trách sẽ phải trả lời đi trả lời lại cùng một đáp án. Từ đây, nảy sinh nhu cầu có một hệ thống có thể tự động tra cứu và giải đáp các thắc mắc của sinh viên dựa trên các tài liệu đã có, nếu không, hệ thống có thể tự động chuyển hướng thắc mắc của sinh viên tới bộ phận hỗ trợ thông qua các kênh truyền thông (Email), mà không cần thiết phải trực tiếp liên lạc với các thầy cô.

Large language model (LLM) đang phát triển, có thể xử lý các bài toán về ngôn ngữ tự nhiên. Các mô hình LLM hiện nay đều là các mô hình học sâu tiên tiến, được huấn luyện với bộ dữ liệu đồ sộ. Do đó, LLM có thể phân tích được các ý định của người dùng và phản hồi lại kết quả dựa trên tri thức được huấn luyện vốn có của nó. Các câu trả lời của LLM đều rất tự nhiên và giống như phản hồi đến từ con người. Ngoài ra, các mô hình LLM hiện nay cũng đều là các mô hình được huấn luyện sẵn, ta chỉ cần gọi và sử dụng chúng, hoặc cấu hình lại mô hình nếu như thực sự cần thiết. Dựa trên sức mạnh đó, ta có thể ứng dụng

các mô hình LLM làm trợ lý ảo hỗ trợ sinh viên, giúp phản hồi sinh viên được hiệu quả và tự nhiên. Tuy nhiên, LLM có nhược điểm là chỉ có thể trả lời những câu hỏi có trong dữ liệu huấn luyện của nó, muốn trả lời các câu hỏi bên ngoài cần phải cập nhật tri thức cho LLM. Có rất nhiều phương pháp cập nhật tri thức cho LLM. Trong khuôn khổ của đề án tốt nghiệp này, tôi quyết định sử dụng phương pháp Retrieval Augmented-Generation (RAG) để giúp LLM có thể trả lời các câu hỏi của sinh viên một cách tự động trên cơ sở các tài liệu và thông tin bên ngoài cung cấp cho LLM.

1.2 Mục tiêu và phạm vi đề án

Trên cơ sở vấn đề cần giải quyết, tôi đặt ra các mục tiêu cần thực hiện đó là:

1. Xây dựng được một hệ thống phản hồi tự động các câu hỏi của sinh viên trường Đại học Giao thông vận tải. Hệ thống sẽ truy xuất thông tin phù hợp với câu hỏi của sinh viên và phản hồi bằng ngôn ngữ tự nhiên một cách tóm lược và ngắn gọn nhất có thể.

2. Trong trường hợp hệ thống không thể có đáp án phản hồi cho sinh viên, hệ thống sẽ tự động chuyển hướng câu hỏi của sinh viên tới địa chỉ email của thầy cô phụ trách vấn đề tương ứng, đồng thời phản hồi thông báo cho sinh viên.

3. Mô hình có thể lấy thông tin trả lời sinh viên từ nguồn tĩnh hoặc tự động tìm kiếm thông tin trên các trang thông tin chính của trường như quản lý đào tạo, website trường.

Phạm vi của đề án sẽ nằm trong các câu hỏi có liên quan đến trường Đại học Giao thông vận tải. Các truy vấn từ sinh viên nằm ngoài phạm vi này sẽ không được hệ thống phản hồi.

1.3 Công nghệ và công cụ sử dụng

1.3.1 Công nghệ sử dụng

Mô hình ngôn ngữ lớn: Mô hình ngôn ngữ lớn là xương sống của hệ thống phản hồi câu hỏi sinh viên tự động. Nó đóng vai trò một người trợ lý có thể trả lời các câu hỏi bằng ngôn ngữ tự nhiên, dễ đọc và dễ hiểu thông qua các tri thức được huấn luyện từ nguồn dữ liệu khổng lồ trên Internet. Các mô hình được tôi lựa chọn bao gồm: Gemini-1.0-pro, GPT-4o, GPT-3.5-turbo.

Tạo sinh truy xuất tăng cường (RAG): RAG là một kỹ thuật cho phép cải thiện khả năng sinh câu trả lời của mô hình bằng cách cung cấp cho mô hình các nguồn tri thức bên ngoài. Các mô hình ngôn ngữ lớn không có khả năng nắm bắt được các thông tin về trường, hoặc nắm bắt vô cùng ít. Mô hình cũng không thể nhận biết được các thông tin thời gian

thực. RAG cho phép ta bổ sung tri thức mà không cần phải huấn luyện hoặc tinh chỉnh mô hình, giúp giảm chi phí và giảm độ phức tạp.

Prompt Engineering: Prompt engineering là một kỹ thuật cho phép tối ưu hóa truy vấn vào mô hình LLM, giúp mô hình LLM có thể hiểu rõ hơn ý đồ của người dùng và sinh câu trả lời đúng và sát nhất, giảm tải được tình trạng mô hình gặp “ảo giác”, sinh câu trả lời sai.

1.3.2 Công cụ sử dụng

Ngôn ngữ lập trình Python: Ngôn ngữ lập trình hướng đối tượng được phát triển lần đầu tiên vào năm 1991 và kể từ đó, nó đã trở thành một trong những ngôn ngữ mạnh mẽ nhất với cú pháp vô cùng dễ học. Ngoài ra, Python được hỗ trợ bởi một cộng đồng đông đảo, có nhiều thư viện hữu ích giúp đẩy nhanh quá trình thực hiện các dự án, đặc biệt là đối với các tác vụ học máy – học sâu và trí tuệ nhân tạo. Do đó, tôi lựa chọn Python làm ngôn ngữ lập trình chính để xây dựng hệ thống tư vấn của mình.

Môi trường phát triển – Google Colab: Google Colab là một môi trường phát triển đám mây dựa trên môi trường phát triển Jupyter Notebook được cung cấp bởi Google. Google Colab vô cùng phổ biến và phù hợp để phát triển các dự án AI – Machine Learning – Deep Learning, do nó cho phép người dùng chạy code Python trực tiếp trên trình duyệt, giảm tải được quá trình cài đặt môi trường, tối ưu hóa các cài đặt, cung cấp tài nguyên phần cứng miễn phí (GPU, TPU), dễ dàng tích hợp với Google Drive, dễ dàng làm việc cùng nhau,...

Một số thư viện được sử dụng trong quá trình xây dựng hệ thống của tôi bao gồm:

Langchain: Langchain là một thư viện cung cấp các API cho phép tương tác với các mô hình ngôn ngữ lớn dễ dàng. Nó cho phép thiết kế các ứng dụng phức tạp có sử dụng LLM mà không cần tốn quá nhiều thời gian cấu hình.

Streamlit: Streamlit là một thư viện giúp xây dựng một ứng dụng web trở nên nhanh chóng và chuyên nghiệp. Streamlit cho phép người dùng thiết kế trang web mà không cần sử dụng HTML, CSS hoặc các framework front-end. Người dùng có thể thiết kế trang web sử dụng Python thông qua các thành phần được cung cấp bởi thư viện này.

Cơ sở dữ liệu lưu trữ vector – Pinecone: Là một nền tảng lưu trữ vector đám mây, Pinecone được biết đến rộng rãi bởi sự đơn giản, miễn phí và dễ dàng thao tác của nó.

CHƯƠNG 2. MÔ HÌNH NGÔN NGỮ LỚN

1 Tổng quan mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn (Large language model – LLM) là xương sống của hệ thống tư vấn chính sách nội quy. LLM đóng vai trò như một người phát ngôn, từ câu hỏi của sinh viên và thông qua quá trình tìm kiếm thông tin liên quan đến câu hỏi, LLM sẽ sinh ra câu trả lời cho sinh viên một cách tự nhiên như một người tư vấn thực thụ.

Trước hết, mô hình ngôn ngữ (Language model) là một mô hình học máy có khả năng dự đoán và sinh ngôn ngữ một cách tự nhiên. Các mô hình này hoạt động bằng cách dự đoán xác suất của một token hoặc một chuỗi các token xuất hiện trong một chuỗi token dài hơn [1].

LLM trên cơ sở là một mô hình ngôn ngữ. Nó được huấn luyện trên một tập dữ liệu rất lớn và đạt được khả năng hiểu biết và sinh ngôn ngữ thông qua các tri thức thu nạp được về cấu trúc và ngữ nghĩa của ngôn ngữ huấn luyện nó. LLM đạt được khả năng này thông qua quá trình huấn luyện và học được các mối quan hệ thống kê trong các tập văn bản thông qua quá trình học tự giám sát và quá trình học bán giám sát [2]. Mô hình ngôn ngữ lớn có thể nhận dạng, tóm tắt, dịch văn bản, dự đoán và sinh nội dung từ tri thức học được thông qua quá trình huấn luyện [3]. Vậy mô hình ngôn ngữ lớn thì "lớn" như thế nào? Để có thể lưu trữ tri thức đã học được từ tập dữ liệu huấn luyện, LLM sử dụng các tham số mô hình. Số lượng tham số của mô hình quyết định độ lớn của mô hình đó.

Bảng 2.1: Bảng thống kê số lượng tham số của một số LLM phổ biến. 1M = 1,000,000 tham số; 1B = 1,000,000,000 tham số; 1T = 10B

Tên mô hình	Số params
BERT	110M
phobert-base-v2	135M
gpt-2	1.5B
gpt-3.5-turbo	20B
gpt-4o	1.76T

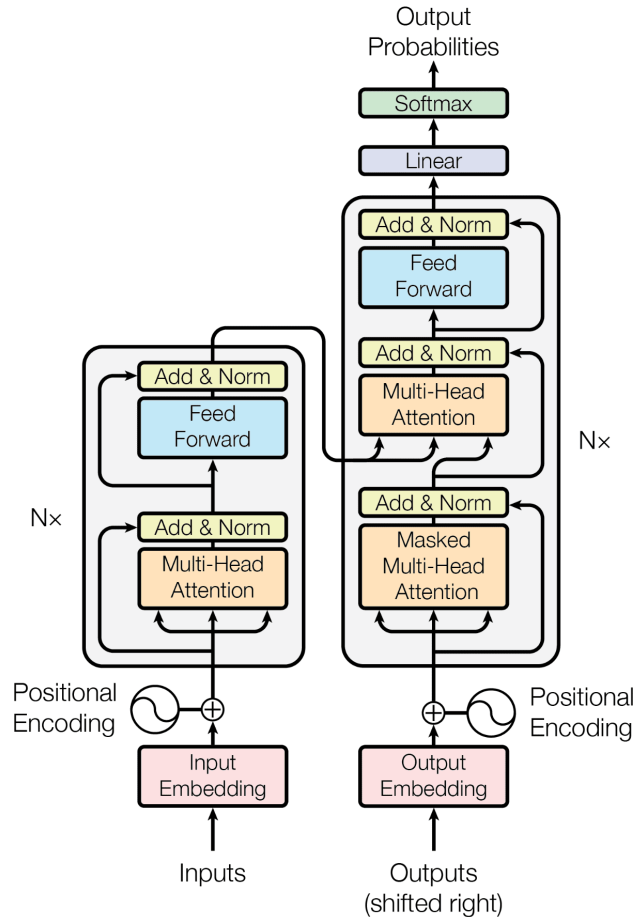
2 Kiến trúc Transformer

2.1 Tổng quan kiến trúc

Transformer là một kiến trúc mô hình giúp giảm tính lặp lại, thay vào đó dựa trên cơ chế attention để phát hiện các cấu trúc phụ thuộc giữa các chuỗi đầu vào và chuỗi đầu ra [4]. Các mô hình lặp lại là các mô hình dựa trên tính lặp lại để tính toán độ quan trọng của một từ (một token – một đơn vị nhỏ nhất có thể tách từ một chuỗi mà vẫn mang ý nghĩa) trong một chuỗi. Tuy nhiên, các mô hình loại này không cho phép việc huấn luyện song song, ngoài ra, khi chuỗi càng dài thì độ mất mát thông tin của các token đầu tiên càng lớn (mô hình quên các token đầu). Mô hình kiến trúc transformer cho phép huấn luyện song song, tận dụng sức mạnh tính toán song song của GPU và cho phép tính toán các mối quan hệ giữa các token trong một chuỗi mà không gây mất mát thông tin.

Mô hình transformer là một mô hình mạng neural học sâu, thường có cấu trúc encoder-decoder, gồm một máy mã hóa (encoder) và một máy giải mã (decoder). Máy mã hóa ánh xạ biểu diễn của chuỗi token đầu vào $\mathbf{x} = (x_1, \dots, x_n)$ thành một chuỗi biểu diễn liên tục $\mathbf{z} = (z_1, \dots, z_n)$, được gọi là chuỗi nhúng của chuỗi $\mathbf{x}$. Từ chuỗi $\mathbf{z}$ máy giải mã sinh chuỗi đầu ra $\mathbf{y} = (y_1, \dots, y_n)$, mỗi lần sinh một phần tử [4]. Mỗi bước của mô hình sẽ tự động hồi quy, sử dụng token vừa được sinh ra trước đó kết hợp với chuỗi đầu vào ban đầu để sinh token tiếp theo. Quá trình dừng lại cho đến khi đạt một điều kiện nào đó, thường là khi mô hình sinh ra token kết thúc ([EOS] - End-Of-Sentence).

2.2 Encoder – Decoder



Hình 2.1 Sơ đồ các lớp neural trong kiến trúc mô hình Transformer. Nguồn: Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).

Một mô hình Transformer điển hình bao gồm 2 bộ phận:

Mã hóa: Một máy mã hóa sử dụng để biến đổi chuỗi đầu vào thành một chuỗi vector số học biểu diễn lại chuỗi đầu vào đó, lưu trữ các thông tin đặc trưng cho chuỗi đầu vào. Một chuỗi mã hóa gồm 1 chồng N lớp giống hệt nhau ($N = 6$ đối với mô hình transformer gốc [4]), mỗi lớp lại có 2 lớp con:

i. *Lớp multi-head attention*: Trích xuất thông tin mối liên hệ giữa các từ với nhau trong chuỗi bằng cách sử dụng cơ chế self-attention để tính toán.

ii. *Lớp Feed-forward (FFN)*: Biến đổi đầu ra của lớp multi-head-attention thành đầu ra cuối cùng của máy mã hóa.

Giữa hai lớp và giữa lớp FFN và đầu ra output có một lớp chuẩn hóa, sử dụng để chuẩn hóa lại các giá trị vector, đưa chúng về cùng một tỉ lệ. Máy mã hóa output ra số chiều là d ($d = 512$ đối với mô hình gốc).

Giải mã: Sử dụng để chuyển chuỗi được mã hóa thành xác suất dự đoán của token tiếp theo trong chuỗi đầu ra. Cũng gồm 1 chồng N lớp giống hệ nhau, mỗi lớp lại có 3 lớp, với hai lớp giống như encoder, bổ sung thêm một lớp multi-head attention để trích xuất thông tin mối quan hệ giữa vector biểu diễn đầu ra của mã hóa với các vector từ ở lớp giải mã.

Các mô hình ngôn ngữ lớn tùy thuộc theo loại mô hình sẽ có các kiến trúc khác nhau. Một số kiểu kiến trúc dạng Transformer phổ biến hiện nay gồm:

i. *Mô hình encoder-only*: Chỉ có máy mã hóa. Phù hợp với các tác vụ chuyển hóa chuỗi văn bản thành vector số học để phục vụ cho các tác vụ phân lớp văn bản hoặc so sánh độ tương đồng, phục vụ trong truy vấn. Có thể coi đây là một quá trình nhúng từ (word-embedding). Một số mô hình nổi bật gồm có: BERT, PhoBERT (đối với tiếng việt).

ii. *Mô hình decoder-only*: Chỉ có máy giải mã. Phù hợp cho các tác vụ sinh văn bản. Một số mô hình tiêu biểu gồm GPT, Llama, Gemini.

iii. *Mô hình encoder-decoder*: Gồm đầy đủ 2 máy mã hóa và giải mã. Phù hợp với các tác vụ text-to-text, chẳng hạn như dịch văn bản hoặc tóm tắt văn bản.

2.3 Cơ chế self-attention

Self-attention là cơ chế cho phép mô hình truy xuất được thông tin mối quan hệ giữa token đang xét (token truy vấn) với các token còn lại trong chuỗi và chính bản thân token đó (các token khóa). Việc trích xuất thông tin này nhằm tính toán độ tương đồng giữa token truy vấn và các token còn lại, nhằm chỉ ra token truy vấn có phải đồng nghĩa với một trong các token khóa hay không. Cuối cùng, giá trị độ tương đồng sẽ được tính toán với toàn bộ các token trong chuỗi (token value) và sinh ra vector biểu diễn token cuối cùng.

Ví dụ, xét câu sau, với các token là các từ được ngăn cách nhau bởi dấu cách

“một nhà_giáo hỏi vị giáo_sư về trí_tuệ nhân_tạo và **ông** đã giải_đáp thắc_mắc của vị nhà_giáo”

Xét token **ông**, token này sẽ liên quan đến “giáo_sư” hay “nhà_giáo”. Thông qua cơ chế self-attention, mô hình sẽ tập trung hơn vào mối quan hệ giữa “ông” và “giáo_sư” hơn là “ông” và các token còn lại trong chuỗi, bảo toàn ngữ nghĩa của các token trong chuỗi.

3 Prompt Engineering

Prompt Engineering là một kỹ thuật khá mới, áp dụng trong việc xây dựng và tối ưu các truy vấn, câu hỏi của người dùng đặt cho các mô hình ngôn ngữ nhằm phục vụ cho các tác vụ rộng hơn [5]. Kỹ thuật này giúp mô hình hiểu tốt hơn ý đồ của người dùng và trả lời thông tin chính xác hơn, tránh tình trạng “ảo giác” của LLM – Mô hình bịa ra câu trả lời để cố gắng trả lời câu hỏi, ngoài ra không cần huấn luyện bổ sung cho mô hình.

Thông thường, khi truy vấn với LLM, người dùng thường đặt ra những câu hỏi vô cùng đơn giản, chẳng hạn:

“Làm sao để quản lý thời gian hiệu quả?”

“Viết một bài luận văn về tác dụng của LLM”

“Sinh một bài quảng cáo cho sản phẩm gia dụng thông minh ABC”

...

Đối với các vấn đề chung chung, hoặc đơn giản người dùng không hề có đáp án hoặc ý định cá nhân nào mà hoàn toàn tham khảo tri thức từ mô hình, các truy vấn đơn giản như trên cũng đã đủ để mô hình sinh câu trả lời hợp lý cho người dùng. Tuy nhiên, khi người dùng muốn câu trả lời từ mô hình phức tạp hơn, hoặc muốn mô hình hiểu ý đồ của người dùng, chẳng hạn muốn mô hình trả lời theo một định dạng nào đó, thì cách đặt câu hỏi chung chung như trên hoàn toàn có thể khiến LLM bịa câu trả lời.

Một số kỹ thuật prompt engineering giúp cải thiện kết quả trả về của mô hình gồm:

Chỉ định hướng dẫn cho prompt: Thay vì ngay lập tức hỏi LLM về vấn đề cần giải quyết, hãy đưa ra các hướng dẫn cho LLM, sau đó dựa trên hướng dẫn để đặt một câu hỏi tương tự cho LLM giải quyết. Ví dụ về một prompt dịch tiếng Anh sang tiếng Việt có áp dụng hướng dẫn:

“Dịch đoạn văn sau từ tiếng Anh sang tiếng Việt một cách tự nhiên và ngắn gọn nhất có thể. Các cụm từ tiếng Anh mà không có từ tiếng Việt sát nghĩa thì hãy dịch thành từ sát nghĩa với ngữ cảnh nhất, sau đó đi kèm với từ tiếng Anh gốc trong cặp dấu ngoặc

Dịch câu sau: Large language model can cause hallucination with simple prompts”

Gán chức vụ cho LLM: Phương pháp này tương tự như chỉ dẫn cho LLM, tuy nhiên LLM được gán một chức vụ nào đó, chẳng hạn như dịch giả, lập trình viên, nhà sáng tạo nội dung,... Phương pháp này giúp mô hình nắm được ngữ cảnh mà nó cần phản hồi thông tin, giúp mô hình tăng cường độ chính xác của phản hồi. Ví dụ với prompt dịch tiếng Anh ở trên:

“Bạn là một dịch giả chuyên nghiệp có thể dịch các đoạn văn bản từ tiếng Anh sang tiếng Việt một cách tự nhiên và ngắn gọn nhất có thể. Các cụm từ tiếng Anh mà không có từ tiếng Việt sát nghĩa thì bạn hãy dịch thành từ sát nghĩa với ngữ cảnh nhất, sau đó đi kèm với từ tiếng Anh gốc trong cặp dấu ngoặc.

Dịch câu sau: Large language model can cause hallucination with simple prompts”

Few-shot Prompting: Prompt sử dụng để truy vấn với LLM thay vì chỉ có chỉ dẫn và yêu cầu mô hình, giờ đây bổ sung thêm một số ví dụ. Việc này giúp mô hình nhận biết được chính xác hơn các thông tin cần sinh ra do ta đã cung cấp được các ví dụ về phản hồi mà mô hình cần phản hồi, do đó nó sẽ hướng đến việc sinh ra phản hồi giống như những ví dụ mà ta cung cấp. Few-shot Prompting rất phù hợp cho các tác vụ phức tạp, nhiều bước, hoặc phản hồi có cấu trúc phức tạp. Ví dụ với đoạn prompt dịch từ Tiếng Anh sang tiếng Việt ở trên:

“Bạn là một dịch giả chuyên nghiệp có thể dịch các đoạn văn bản từ tiếng Anh sang tiếng Việt một cách tự nhiên và ngắn gọn nhất có thể. Các cụm từ tiếng Anh mà không có từ tiếng Việt sát nghĩa thì bạn hãy dịch thành từ sát nghĩa với ngữ cảnh nhất, sau đó đi kèm với từ tiếng Anh gốc trong cặp dấu ngoặc.

Ví dụ 1:

Dịch câu sau: Large language model can cause hallucination with simple prompts

AI: Mô hình ngôn ngữ lớn có thể xảy ra ảo giác (hallucination) với những lệnh (prompts) đơn giản.

Ví dụ 2:

Dịch câu sau: Large language model is built based on the transformer architecture

AI: Mô hình ngôn ngữ lớn được xây dựa trên kiến trúc transformer.

Dịch câu sau: Large language model can cause hallucination with simple prompts”

CHƯƠNG 3. TRUY VẤN TẠO SINH TĂNG CƯỜNG

3.1 Tổng quan về RAG

Trên thực tế, các LLM thường đã được huấn luyện trước và đã được tinh chỉnh để phù hợp cho quá trình sử dụng của người dùng. Đó là lý do tại sao, các chatbot như ChatGPT hoặc Gemini lại đưa ra câu trả lời vô cùng tự nhiên và dễ hiểu đối với con người. Tuy nhiên, lượng tri thức của mỗi mô hình LLM lại có giới hạn, do chúng chỉ có thể thu thập tri thức từ một tập dữ liệu huấn luyện cố định. Chẳng hạn, mô hình GPT-3.5-turbo-0125 (Mô hình GPT-3.5 mới nhất được giới thiệu bởi OpenAI) chỉ giới hạn tri thức đến trước tháng 9 năm 2021 [6], tương đối cũ. Do đó, các mô hình LLM không đáng tin cậy đối với các câu hỏi mang tính thời điểm của người dùng. Mặt khác, do chi phí và thời gian, một mô hình LLM không thể liên tục cập nhật dữ liệu huấn luyện của nó và huấn luyện lại. Do đó, cần phải có một phương pháp giúp mô hình LLM tận dụng được khả năng sinh văn bản của nó, đồng thời cập nhật tri thức cho mô hình theo thời gian.

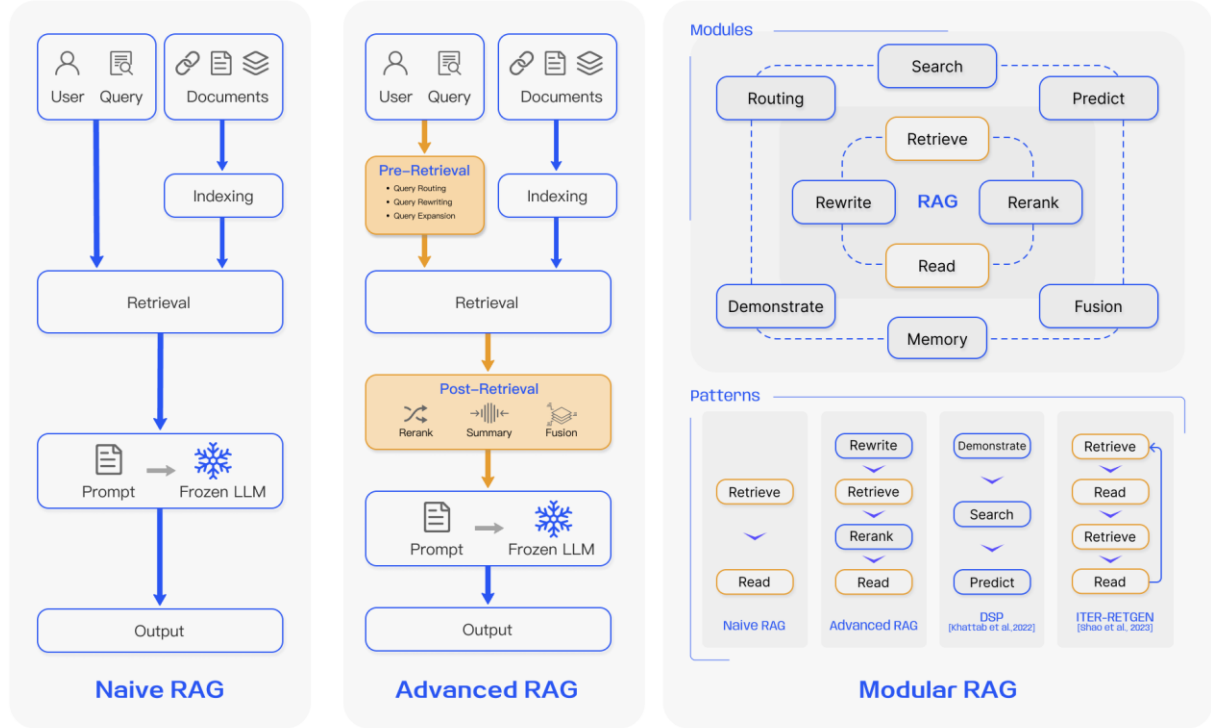
Truy vấn tạo sinh tăng cường (Retrieval Augmented-Generation – RAG) được giới thiệu lần đầu bởi các nhà nghiên cứu của MetaAI vào năm 2020 [7]. RAG kết hợp mô hình LLM với các tri thức bên ngoài, giúp mô hình sinh văn bản chính xác hơn, mang tính kịp thời và giúp giảm thiểu việc mô hình bị “ảo giác”.

Theo Lewis et al., (2021), một mô hình RAG là một mô hình sử dụng một chuỗi đầu vào x để truy vấn các tài liệu dạng văn bản z và sử dụng z như một ngữ cảnh thông tin bổ sung cho mô hình trong quá trình sinh văn bản [8]. RAG nhận đầu vào là một chuỗi truy vấn và sử dụng nó để truy vấn ra một tập hợp các tài liệu văn bản có liên quan đến chuỗi truy vấn đó từ các nguồn xác định (file tĩnh, website hoặc các CSDL). Các tài liệu văn bản có liên quan sau đó được kết hợp lại thành ngữ cảnh bổ sung cho mô hình, được truyền vào mô hình cùng với câu hỏi truy vấn của người dùng, sử dụng kỹ thuật prompt engineering để tạo nên truy vấn hoàn chỉnh để gửi tới LLM. LLM sau đó sẽ sinh câu trả lời dựa trên ngữ cảnh đã đưa cho nó. RAG cho phép mô hình không cần phải huấn luyện lại, do đó giảm chi phí, lại cho phép cập nhật thông tin bổ sung cho mô hình vô cùng nhanh chóng.

Phương pháp RAG đã được kiểm nghiệm trên rất nhiều bài kiểm nghiệm, chẳng hạn như Natural Questions [9], WebQuestions [10] và TrecQA [11]. Do sự đơn giản của RAG, chỉ cần một vài dòng code, ta hoàn toàn có thể xây dựng một hệ thống RAG cho kết quả phản hồi người dùng vô cùng hiệu quả. Do đó, với nhu cầu xây dựng hệ thống tư vấn cho sinh viên trường Đại học Giao thông Vận tải, trên cơ sở quy mô tài liệu dựa vào chưa quá lớn, tôi quyết định sử dụng RAG là phương pháp chính để thực hiện đề tài.

3.2 Phân loại RAG

Phương pháp RAG gồm 3 dạng chính: RAG ngây thơ (Naïve RAG), RAG nâng cao (Advanced RAG) và RAG Modular.



Hình 3.1: So sánh giữa ba dạng RAG chính. Nguồn: Gao, Yunfan et al. “Retrieval-Augmented Generation for Large Language Models: A Survey.” *ArXiv abs/2312.10997* (2023) [12].

RAG Ngây thơ: Phương pháp RAG đơn giản nhất. Truy vấn của người dùng được sử dụng để truy vấn các văn bản liên quan được lưu trữ thông qua quá trình indexing. Các văn bản liên quan được sử dụng làm ngữ cảnh bổ sung, kết hợp với truy vấn của người dùng gốc để tạo ra prompt đầu vào cho LLM. LLM sau đó sinh ra kết quả cho người dùng.

RAG Nâng cao: Tương tự như RAG Ngây thơ, tuy nhiên khắc phục một số hạn chế của RAG Ngây thơ bằng cách bổ sung thêm 2 quy trình nhỏ:

i. *Pre-retrieval*: Tập trung vào việc tối ưu hóa cấu trúc indexing và truy vấn gốc. Mục tiêu của việc tối ưu hóa đó là cải thiện chất lượng của văn bản được indexing. Một số phương pháp tối ưu gồm có: Tối ưu hóa cấu trúc index, bổ sung metadata, biến đổi truy vấn gốc,... [12].

ii. *Post-retrieval*: Tập trung vào tối ưu hóa quá trình tích hợp tri thức bên ngoài đã được tìm kiếm với câu hỏi gốc của người dùng. Phương pháp chính của quy trình này đó là re-ranking và nén tri thức bên ngoài [12].

RAG Modular: Tập trung vào tính nâng cao hơn hai phương pháp trên và tính linh hoạt. Ở phương pháp này, ta bổ sung thêm một số module hỗ trợ cho quá trình RAG, do đó giúp RAG hỗ trợ tốt hơn so với việc chỉ dựa vào tri thức bên ngoài [12].

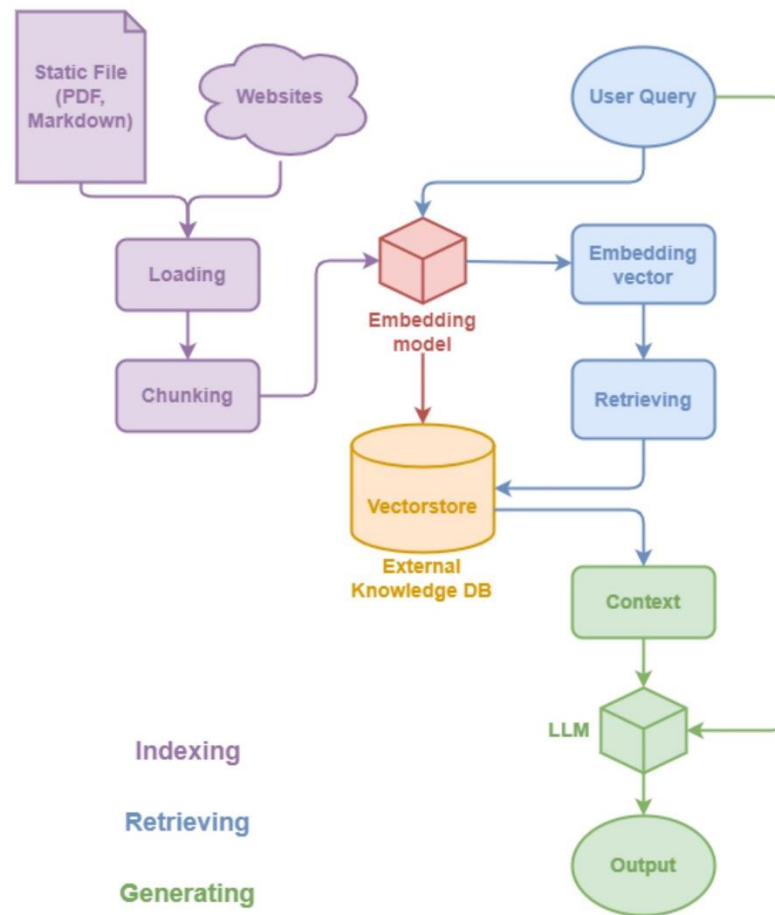
3.3 Quy trình thực hiện phương pháp RAG ngây thơ (Naive RAG)

Quá trình thực hiện kỹ thuật RAG có thể chia làm 3 giai đoạn:

i. **Indexing**: Chuẩn bị và chuyển hóa các tri thức cập nhật cho LLM thành các vector số học và lưu trữ trong một CSDL đặc biệt phục vụ cho quá trình truy vấn.

ii. **Retrieving**: Từ truy vấn của người dùng, thông qua việc tính toán độ tương đồng giữa truy vấn này và các phân đoạn thuộc tri thức bổ sung, truy xuất ra các tài liệu liên quan để cung cấp cho LLM.

iii. **Generating**: Sử dụng câu truy vấn từ người dùng kết hợp với ngữ cảnh tri thức bổ sung để tạo prompt truyền vào mô hình.



Hình 3.2: Quy trình thực hiện kỹ thuật RAG Naive

3.2.1 Indexing

Indexing là quá trình tổ chức lại dữ liệu (tri thức bổ sung cho LLM) sao cho quá trình truy vấn ở giai đoạn Retrieving được nhanh chóng và chính xác nhất. Tổ chức dữ liệu hiệu quả giúp tối ưu kết quả tìm kiếm tài liệu liên quan đến truy vấn, giảm số lượng tài liệu cần truy vấn, tăng độ chính xác của tài liệu cần tìm. Khi này, LLM chỉ cần nạp vào một prompt đơn giản và ngắn hơn, giúp tiết kiệm chi phí. Ví dụ, đối với các mô hình OpenAI, ngoài chi phí cho mỗi lần truy vấn, người dùng cũng bị giới hạn số chi phí tối đa có thể truy vấn trong một ngày (\$120 đối với gpt-4o)

Bảng 3.1: Bảng chi phí tiêu tốn khi sử dụng một số mô hình của OpenAI. Nguồn: <https://openai.com/api/pricing/>

Model	Input	Output
-------	-------	--------

gpt-4o	\$5.00 / 1M tokens	\$15.00 / 1M tokens
gpt-4o-2024-05-13	\$5.00 / 1M tokens	\$15.00 / 1M tokens
gpt-3.5-turbo-0125	\$0.50 / 1M tokens	\$1.50 / 1M tokens
gpt-3.5-turbo-instruct	\$1.50 / 1M tokens	\$2.00 / 1M tokens
text-embedding-3-small	\$0.02 / 1M tokens	

Có thể nói, indexing là quá trình chuẩn bị và tái biểu diễn các thông tin bổ sung cho LLM phục vụ truy vấn.

Quá trình Indexing gồm các giai đoạn sau:

Loading: Các tài liệu tri thức bổ sung cho LLM thường được lưu trữ dưới nhiều định dạng. Các định dạng phổ biến bao gồm các file PDF, file markdown, file docx hoặc file html. Ngoài ra, tri thức bổ sung có thể đến từ website hoặc các API gọi từ các dịch vụ có sẵn, trả về dưới định dạng JSON. Do đó, cần phải đọc và tổ chức các tài liệu này một cách hiệu quả. Các tài liệu này sau đó, để dễ dàng cho việc lưu trữ, phải được chuyển thành dạng chữ thuần.

Chunking: Chunking là việc tách các văn bản thuần đã có sau quá trình loading thành cách phân đoạn nhỏ hơn được gọi là các chunk. Các chunk này thường có một độ dài không vượt quá một giá trị nào đó. Lý do là bởi cửa sổ truy vấn của LLM thường có hạn, tức là số lượng token có thể nhận của một LLM có giới hạn. Việc chunking gần như quyết định rất lớn đến hiệu quả của việc chunking.

Chunking thường sử dụng 2 tham số chính để quyết định tính chất của một chunk được tách. Đó là số lượng ký tự tối đa của chunk (`chunk_size`) và số lượng ký tự trùng lặp nhau giữa hai chunk liền kề (`chunk_overlap`). Khi chunking, ta phải quan tâm đến hai tham số này kỹ càng:

- Nếu `chunk_size` quá nhỏ, lượng thông tin chứa trong một chunk quá ít, dẫn đến thông tin của chunk có thể vô nghĩa hoặc mất nghĩa. Ngược lại, nếu `chunk_size` quá lớn, độ lớn của chunk lớn dẫn đến không còn ý nghĩa của quá trình chunking.

- `chunk_overlap` quyết định độ liên quan giữa hai chunk liền kề. Giả sử nếu một chunk đã đạt đến `chunk_size` tối đa mà cắt ngang một câu nào đó, thì câu đó sẽ bị hụt mất thông tin. `chunk_overlap` giúp ta lặp lại các tri thức, giúp bảo toàn ý nghĩa của câu ở chunk tiếp theo, khi truy vấn thì hai chunk này sẽ được nạp vào LLM.

Một số kỹ thuật chunking bao gồm:

i. *Chunking với size cố định*: Phương pháp đơn giản và dễ dàng nhất. Ta sẽ tách các văn bản thành các chunk sao cho đảm bảo kích thước chunk_size và chunk_overlap, không quan tâm đến cấu trúc của văn bản [14].

ii. *Recursive Chunking*: Chia văn bản thành từng chunk dựa trên cấu trúc của văn bản tuân theo các dấu câu. Nếu chunk chưa đạt kích thước chunk_size hoặc chưa đạt cấu trúc mong muốn thì hàm chunking sẽ tự gọi đệ quy trên các chunk vừa tách, sao cho đạt được kích thước chunk_size hoặc đạt được cấu trúc của mong muốn. Cấu trúc ở đây có thể là làm sao tách trọn vẹn được n câu, tách trọn vẹn được 1 đoạn,... Khi này, các chunk sẽ có độ dài không giống nhau. Tuy vậy, hàm chunking sẽ cố gắng để các chunk có độ dài như nhau [14].

iii. *Chunking theo cấu trúc văn bản*: Một số văn bản được viết dưới các định dạng markdown, JSON hoặc HTML, là các dạng văn bản có cấu trúc. Do đó, có thể sử dụng phép tách theo cấu trúc để bảo toàn các cấu trúc ban đầu của văn bản [14].

Embedding: Các chunk được tách từ bước chunking sẽ được truyền vào một mô hình nhúng từ. Mô hình nhúng từ sẽ chuyển hóa các chunk văn bản thành các vector số học. Các vector này được sử dụng để tính toán độ tương đồng giữa chunk văn bản và câu truy vấn của người dùng.

Storing: Các vector nhúng, các chunk văn bản và các siêu thông tin (metadata) của các chunk văn bản sẽ được kết hợp thành một bản ghi và được gán cho một chỉ mục (index) nào đó, đóng vai trò như khóa chính của chunk đó. Các bản ghi này sau đó được lưu trữ trong một CSDL được gọi là vector store. Vector store này sẽ hỗ trợ quá trình truy vấn các văn bản liên quan đến truy vấn của người dùng, lưu trữ lâu dài và tổ chức các bản ghi sao cho có thể thực hiện nhiều truy vấn trong tương lai mà không cần phải thực hiện lại các bước loading, chunking và embedding từ đầu.

3.2.2 Nhúng từ (Embedding)

Trong các bài toán NLP, nhúng từ (Word-embedding) hay nhúng (embedding) là một phương pháp nhằm biểu diễn lại văn bản thành các vector giá trị thực, mã hóa ý nghĩa của các văn bản sao cho khi biểu diễn trong không gian số học, các văn bản có ý nghĩa tương tự nhau sẽ có vị trí gần nhau [15]. Các vector giá trị thực này được tính toán thông qua việc ánh xạ các token với các giá trị thực thông qua các mô hình ngôn ngữ được gọi là các mô hình nhúng (embedding-model).

Thông qua việc nhúng từ, các văn bản, vốn không thể xử lý trực tiếp bởi các mô hình học máy – học sâu, có thể được chuyển hóa thành dạng mà mô hình học máy – học sâu có thể hiểu. Ngoài ra, việc nhúng từ còn giúp bảo toàn ngữ nghĩa của văn bản bằng các giá trị thực, do đó để xác định các câu tương đồng nhau, ta có thể sử dụng các phương pháp tính toán số học.

Một vector nhúng thường có số chiều rất nhiều. Thông thường, số chiều càng lớn, lượng thông tin về ngữ nghĩa của văn bản đó càng được bảo toàn. Khi này, tính toán độ tương đồng sẽ mang lại độ chính xác cao hơn. Có thể hiểu đơn giản quá trình nhúng từ là quá trình ta nén ngữ nghĩa của văn bản thành các vector nhiều chiều, mỗi chiều mang một đặc trưng ngữ nghĩa nào đó của văn bản.

Bảng 3.2: Số lượng token tối đa có thể nhúng và số chiều của vector nhúng đầu ra của một số mô hình embedding phổ biến

Tên mô hình	Số token tối đa đầu vào	Số chiều vector nhúng
vinai/phobert-base-v2	256	768
openai/text-embedding-3-large	8191	3072
google/embedding-001	2,048	768

3.2.3 Retrieving

Retrieving là quá trình truy vấn các tài liệu liên quan đến truy vấn của người dùng cho LLM. Các tài liệu liên quan này đóng vai trò như tri thức bổ sung bên ngoài cho LLM.

Quy trình thực hiện retrieving gồm các bước:

Embedding: Truy vấn của người dùng, thường là văn bản thuần, sẽ được truyền vào mô hình nhúng từ và chuyển thành vector nhúng. Ta sẽ sử dụng lại mô hình nhúng từ của quá trình indexing, nhằm đảm bảo các vector nhúng của truy vấn sẽ giữ được tính chất giống với vector nhúng được lưu trong vector store, đảm bảo quá trình truy vấn.

Query (Retrieving): Vector nhúng của truy vấn người dùng được truy vấn với tất cả các vector nhúng trong vector store. Thông qua các phép tính toán độ tương đồng, vector store sẽ trả về k văn bản liên quan nhất đến truy vấn của người dùng. k thường được chọn là 5 hoặc 10, nhằm tối giản số lượng token của prompt đầu vào của LLM.

3.2.4 Generating

Generating là quá trình sinh câu trả lời từ truy vấn của người dùng kết hợp với các tri thức liên quan truy vấn được bởi LLM. Đây sẽ là bước quyết định kết quả phản hồi cho người dùng.

Generating gồm các bước sau:

Prompting (context): Từ các văn bản liên quan, kết hợp với truy vấn của người dùng, ta sử dụng kỹ thuật prompt engineering để xây dựng nên prompt đầu vào của LLM. Prompt này thường có cấu trúc như sau:

“ < *system instruction* >

< *context* >

< *question* > ”

, trong đó:

- < *system instruction* >: Là chỉ dẫn mà LLM phải thực hiện. Thường ta sẽ gán LLM với một vai trò nào đó, sau đó đi kèm những việc mà LLM phải làm để phản hồi truy vấn và những việc LLM sẽ làm nếu như không thể phản hồi truy vấn của người dùng.

- < *context* >: Các văn bản liên quan đến truy vấn của người dùng đã có từ bước retrieving.

- < *question* >: Truy vấn gốc của người dùng.

Ví dụ về một prompt được sinh ra để truyền vào LLM:

“Bạn là một chatbot hỗ trợ trả lời các câu hỏi của người dùng dựa trên các thông tin được cung cấp dưới đây. Nếu không thể tìm thấy thông tin phù hợp cho câu hỏi của người dùng, hãy trả lời: ‘Tôi không biết’.

Thông tin ngữ cảnh cung cấp: {*context*}

Câu hỏi: {*question*}”

, với {*context*} và {*question*} được bổ sung trong quá trình chạy thực tế.

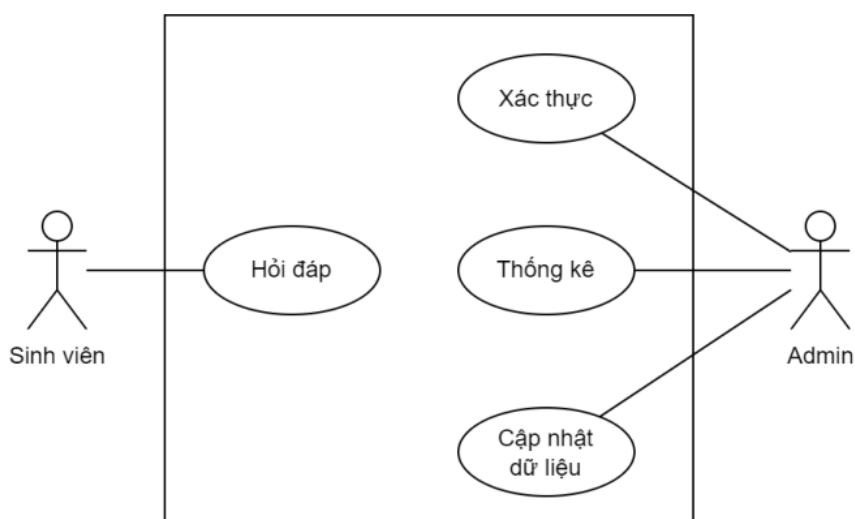
Việc câu trả lời tự nhiên hay không, câu trả lời có ngắn gọn hay không, hoặc câu trả lời có đúng ý đồ của người quản trị hệ thống hay không phụ thuộc vào cách xây dựng prompt đầu vào của mô hình. Chú ý rằng, một số mô hình LLM sẽ tính phí dựa trên số lượng token của prompt đầu vào. Nếu prompt quá dài, chi phí sẽ tăng cao, thậm chí số lần truy vấn cũng bị giới hạn.

LLM Generating: Prompt được sinh ra sau quá trình prompting sẽ được truyền vào LLM. LLM sau đó sẽ sinh ra câu trả lời đầu ra cho người dùng.

CHƯƠNG 4. XÂY DỰNG HỆ THỐNG TƯ VẤN NỘI QUY TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI

4.1 Phân tích thiết kế hệ thống

4.1.1 Sơ đồ ca sử dụng



Hình 4.1: Sơ đồ ca sử dụng của hệ thống tư vấn nội quy sinh viên trường Đại học Giao thông vận tải.

Hệ thống tư vấn nội quy sinh viên trường Đại học Giao thông vận tải của tôi sẽ gồm 2 trang web chính: Trang web tư vấn viên và trang web quản trị tư vấn nội quy. Trang web tư vấn viên chính là trang web nơi người dùng trực tiếp đặt câu hỏi và nhận phản hồi từ LLM; Trang web quản trị tư vấn nội quy là trang web nơi người quản trị quản lý LLM và cập nhật các tri thức cho LLM cũng như các thiết lập phục vụ cho quá trình truy vấn từ người dùng.

Đặc tả các chức năng chi tiết được trình bày ở bảng 4.1 dưới đây:

Bảng 4.1: Mô tả các chức năng có trong hệ thống tư vấn nội quy trường Đại học Giao thông vận tải

Mã chức năng	Tên chức năng	Mô tả
--------------	---------------	-------

UC01	Hỏi đáp	Sinh viên đặt câu hỏi trực tiếp đến hệ thống. Mô hình tìm kiếm thông tin phù hợp để trả lời câu hỏi của sinh viên và hiển thị phản hồi cho người dùng
UC02	Xác thực	Đăng nhập quyền quản trị ở trang quản trị. Chỉ có số lượng giới hạn tài khoản có thể đăng nhập
UC03	Thống kê	Thống kê các số liệu các câu hỏi của sinh viên, gồm có thống kê số lượng câu hỏi trong ngày, số lượng câu hỏi đã nhận, chi phí, biểu đồ tăng trưởng câu hỏi, chủ đề sinh viên quan tâm
UC04	Cập nhật dữ liệu	Cập nhật tri thức ngoài cho mô hình phục vụ cho việc mô hình truy vấn thông tin để trả lời câu hỏi từ sinh viên

4.1.2 Đặc tả các chức năng

4.1.2.1 UC01 – Hỏi đáp

Bảng 4.2: Đặc tả chức năng UC01 - Hỏi đáp

Use case: UC01 – Hỏi đáp	
Mục đích	Người dùng hỏi đáp các thông tin về nội quy trường Đại học Giao thông vận tải
Tác nhân	Người dùng
Mô tả	Người dùng hỏi các thông tin về nội quy của trường. Sau đó, hệ thống phản hồi lại các nội dung tương ứng với câu hỏi của người dùng. Người dùng có thể hỏi nhiều thông tin cùng một lúc, hệ thống sẽ ghi nhớ và tiếp tục phản hồi lại người dùng. Người dùng không cần đăng nhập. Người dùng có thể chọn các tư vấn viên phù hợp với chủ đề cần hỏi của mình
Tiền điều kiện	Không có

Quy trình thực hiện	Phía người dùng: 1. Người dùng lựa chọn tư vấn viên phù hợp 2. Người dùng nhập câu hỏi cần giải đáp 5. Người dùng nhập câu hỏi tiếp theo cần giải đáp và quay lại bước 1. Ngược lại, người dùng thoát hệ thống. Phía hệ thống: 3. Hệ thống nhận câu hỏi của người dùng. 4. Hệ thống phản hồi lại câu trả lời cho người dùng
Hậu điều kiện	Không có

4.1.2.2 UC02 – Xác thực

Bảng 4.3: Đặc tả chức năng UC02 - Xác thực

Use case: UC02 – Xác thực	
Mục đích	Xác thực người dùng có phải người quản trị hệ thống tư vấn hay không
Tác nhân	Người quản trị
Mô tả	Để truy cập trang quản trị, người quản trị phải đăng nhập bằng tài khoản quản trị. Chỉ có duy nhất một tài khoản quản trị được chia sẻ giữa các người quản trị. Người quản trị có quyền thay đổi mật khẩu tài khoản, xem số dư chi phí của mô hình.
Tiền điều kiện	Người dùng truy cập trang quản trị

Quy trình thực hiện	Phía người dùng: 1. Người dùng truy cập trang quản trị 3. Người dùng nhập tài khoản và mật khẩu của tài khoản quản trị Phía hệ thống: 2. Hệ thống hiển thị trang đăng nhập 4. Hệ thống kiểm tra tài khoản của người dùng nhập 4.1. Nếu tài khoản hợp lệ, hệ thống chuyển hướng người dùng đến trang quản trị 4.2. Nếu tài khoản không hợp lệ, hệ thống báo lỗi cho người dùng.
Hậu điều kiện	Không có

4.1.2.3 UC03 – Thống kê

Bảng 4.4: Đặc tả chức năng UC03 - Thống kê

Use case: UC03 – Thống kê	
Mục đích	Thống kê các thông số liên quan đến các câu hỏi của người dùng
Tác nhân	Người quản trị
Mô tả	Hệ thống hiển thị các số liệu cho người quản trị bao gồm: Số lượng câu hỏi trong ngày, tổng số lượng câu hỏi, chi phí phát sinh trong ngày, tổng chi phí đã phải trả, biểu đồ biểu diễn độ tăng giảm số câu hỏi, biểu đồ biểu diễn độ tăng giảm theo chủ đề câu hỏi, biểu đồ thống kê số lượng câu hỏi theo chủ đề.
Tiền điều kiện	Người dùng đã đăng nhập tài khoản quản trị
Quy trình thực hiện	Phía người dùng: 1. Người dùng truy cập trang thống kê Phía hệ thống: 2. Hệ thống hiển thị trang thống kê kèm các số liệu

Hậu điều kiện	Không có
---------------	----------

4.1.2.4 UC04 – Cập nhật dữ liệu

Bảng 4.5: Đặc tả chức năng UC04 - Cập nhật dữ liệu

Use case: UC04 – Cập nhật dữ liệu	
Mục đích	Cập nhật các thông tin liên quan đến CSDL tri thức ngoài và cập nhật tri thức ngoài bổ sung cho mô hình.
Tác nhân	Người quản trị
Mô tả	Người dùng có thể thay đổi hoặc xem thông tin liên quan đến CSDL tri thức ngoài hoặc truy vấn thử các câu hỏi để kiểm tra. Người dùng có thể cập nhật thêm các tri thức bổ sung cho mô hình vào CSDL.
Tiền điều kiện	Người dùng đã đăng nhập tài khoản quản trị
Quy trình thực hiện	Phía người dùng: 1. Người dùng truy cập trang cập nhật thông tin 3. Người dùng truy cập mục CSDL 5. Người dùng truy cập mục cập nhật thông tin Phía hệ thống: 2. Hệ thống hiển thị trang cập nhật thông tin 4. Hệ thống hiển thị các thông tin về CSDL 6. Hệ thống hiển thị trang cập nhật thông tin
Hậu điều kiện	Không có

4.2 Xây dựng trang web tư vấn viên nội quy

4.2.1 Cài đặt chatbot sử dụng phương pháp RAG

4.2.1.1 Indexing

Trước hết, nguồn dữ liệu là tri thức bổ sung cho LLM, trọng tâm sẽ là quyền sách sổ tay sinh viên. Tôi lựa chọn đây là nguồn dữ liệu nền tảng cho LLM. Vậy lựa chọn tri thức nền tảng cho LLM như thế nào? Tôi dựa vào các tính chất sau của dữ liệu:

- i. *Tính chính danh*: Dữ liệu đã được xác thực và có uy tín liên quan đến trường.
- ii. *Tính phổ quát*: Dữ liệu cần phải phổ quát, tổng quan và chứa các tri thức bao phủ tất cả các vấn đề liên quan đến trường Đại học Giao thông vận tải. Nó đồng thời cũng phải tối giản để đảm bảo cô đọng tri thức cho sinh viên.
- iii. *Tính ổn định*: Tài liệu phải ổn định, ít thay đổi nội dung quá nhiều trong thời gian dài.

Sổ tay sinh viên là một tài liệu rất phù hợp, thỏa mãn đủ các tiêu chí trên. Trước hết, nó do chính trường Đại học Giao thông vận tải lưu hành xuyên suốt các thế hệ sinh viên, vô cùng phổ biến và cô đọng, chứa đầy đủ các thông tin để sinh viên có thể tra cứu và tham khảo. Ngoài ra, nội dung trong sổ tay sinh viên ít bị thay đổi quá nhiều qua các khóa, rất ổn định. Tài liệu nền tảng này sẽ được lưu trữ trong vector store đầu tiên, sau đó, có thể bổ sung các tài liệu khác thông qua trang quản trị tư vấn nội quy. Mục đích là tạo cơ sở dữ liệu uy tín cho LLM để xử lý các câu hỏi đơn giản và phổ thông của sinh viên.

Do sổ tay sinh viên được lưu trữ dưới dạng file PDF, do đó, tôi sử dụng PyPDF, một thư viện miễn phí của Python để đọc file này. Langchain cung cấp API cho phép tôi tích hợp chức năng của PyPDF để đọc file đơn giản hơn. File được đọc từ PyPDF sẽ được tổ chức thành một danh sách gồm n đối tượng Document, với n là số trang của tài liệu. Mỗi đối tượng Document này lưu trữ các thông tin gồm: Nội dung được đọc (page_content), siêu thông tin về trang (metadata). Chú ý nội dung của trang sẽ được chuyển hoàn toàn thành văn bản thông thường, và không chứa hình ảnh.

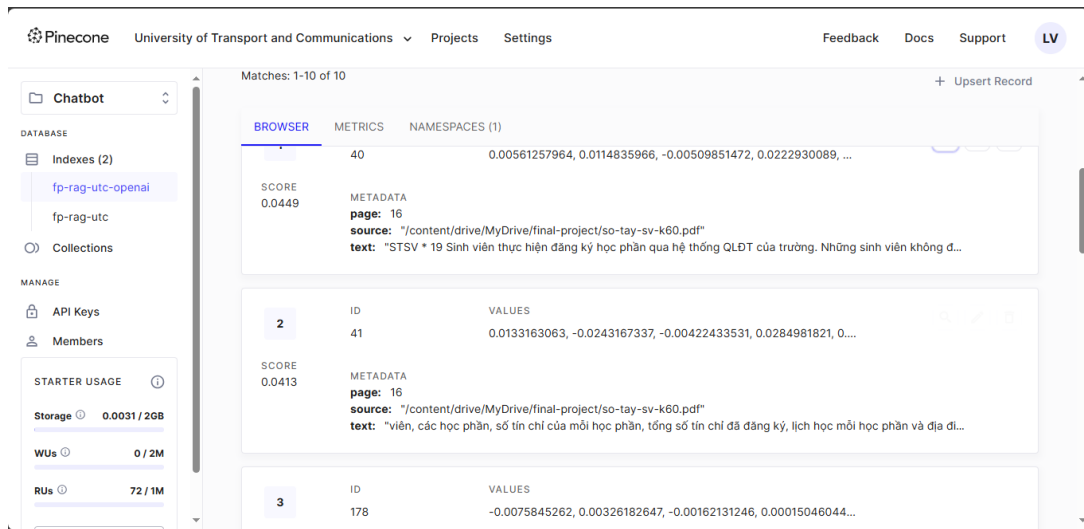
```
Document(page_content=' \nSTSV * 3 MỤC LỤC LỜI NÓI ĐẦU 05 PHẦN 1: GIỚI THIỆU VỀ TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI 07 1. Giới thiệu chung 07 2. Đội ngũ, cơ cấu tổ chức 08 3. Các kênh thông tin online dành cho sinh viên 09 4. Địa chỉ, điện thoại các đơn vị thực hiện công tác sinh viên 10 PHẦN 2: CÁC QUY CHẾ, QUY ĐỊNH 12 PHẦN 2.1. QUY CHẾ ĐÀO TẠO ĐẠI HỌC 12 PHẦN 2.2. QUY ĐỊNH VỀ CHUẨN ĐẦU RA NGOẠI NGỮ 35 PHẦN 2.3. QUY ĐỊNH VỀ ĐÁNH GIÁ RÈN LUYỆN SINH VIÊN 38 PHẦN 2.4. QUY ĐỊNH VỀ KỶ LUẬT SINH VIÊN 40 PHẦN 2.5. QUY ĐỊNH VỀ HỌC BỔNG 42 PHẦN 2.6. QUY ĐỊNH VỀ CHẾ ĐỘ CHÍNH SÁCH 44 PHẦN 2.7. QUY ĐỊNH VỀ VAY VỐN TÍN DỤNG 49 PHẦN 2.8. QUY ĐỊNH QUẢN LÝ SINH VIÊN NGOẠI TRÚ 50 PHẦN 3: HƯỚNG DẪN THỰC HIỆN 52 I. TÀI KHOẢN QUẢN LÝ ĐÀO TẠO VÀ ĐĂNG KÝ HỌC 52 ', metadata={'source': '/content/drive/MyDrive/final-project/so-tay-sv-k60.pdf', 'page': 0})
```

Hình 4.2: Ví dụ về một đối tượng Document được đọc bằng PyPDF từ sổ tay sinh viên

Đối với quá trình chunking: tôi sử dụng chiến lược Recursive chunking, với $\text{chunk_size} = 1000$ và $\text{chunk_overlap} = 500$. Tức là với mỗi một chunk, số lượng ký tự tối đa của nó là 1000 ký tự và giữa hai chunk liền kề sẽ có 500 ký tự trùng nhau. Tổng số chunk thu được là 240 chunk.

Đối với quá trình nhúng từ: Tôi sử dụng mô hình nhúng từ text-embedding-3-large của OpenAI. Mô hình có số lượng token tối đa đầu vào rất lớn, đồng thời số chiều của vector đầu ra là rất nhiều (3072 chiều). Do đó, text-embedding-3-large sẽ đủ mạnh để nhúng từ, tạo ra các vector lưu trữ được nhiều đặc trưng ngữ nghĩa của các chunk nhất.

Đối với quá trình storing: Tôi sử dụng Pinecone làm vector store cho hệ thống. Pinecone là một hệ thống cung cấp bộ nhớ lâu dài cho các ứng dụng AI với hiệu năng cao. Pinecone còn là một nền tảng quản lý CSDL vector thuần lưu trữ đám mây, cung cấp các API có sẵn giúp tương tác với CSDL và hỗ trợ truy vấn [16]. Do đó, tôi sử dụng Pinecone làm nền tảng lưu trữ vector của mình. Do Pinecone lưu trữ vector trên bộ nhớ đám mây, do đó dễ dàng truy cập và không tốn chi phí cho việc lưu trữ tại máy cục bộ.



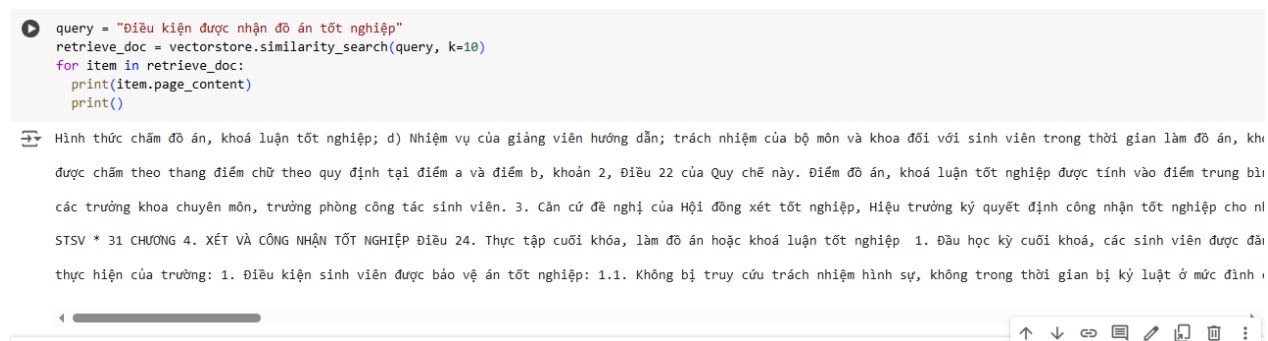
Hình 4.3: Giao diện trang quản trị CSDL vector trên Pinecone

Kết quả của quá trình indexing, số lượng vector được lưu trữ là 240 vector (3072 chiều), tương ứng với 240 chunk văn bản, mỗi chunk văn bản có độ dài tối đa 1000 ký tự, $\text{chunk_overlap} = 500$ ký tự. Nguồn tài liệu là sổ tay sinh viên trường Đại học Giao thông vận tải.

4.2.1.2 Retrieving & Generating

Đối với quá trình retrieving, tôi thực hiện những câu hỏi của người dùng thành vector giá trị thực, sử dụng cùng mô hình text-embedding-3-large giống với quá trình

indexing. Sau đó, tôi thực hiện truy vấn sử dụng API của Pinecone, truyền vào vector nhúng vừa tạo. Số lượng truy vấn trả về được tôi thiết lập là 10 tài liệu.



Hình 4.4: Kiểm thử quá trình trả về tài liệu truy vấn từ Pinecone với câu hỏi truy vấn là "Điều kiện được nhận đồ án tốt nghiệp", số tài liệu trả về là 10. Chỉ nội dung của văn bản liên quan được in ra, mỗi văn bản cách nhau bởi một dòng.

Từ kết quả truy vấn rất khả quan, tôi tiến hành xây dựng mẫu prompt để làm đầu vào cho mô hình. Mẫu prompt có nội dung dưới đây:

"Bạn là một nhân viên hỗ trợ sinh viên của trường Đại học Giao thông vận tải với nhiệm vụ trả lời các thắc mắc đến từ sinh viên của trường. Trước hết, bạn phải luôn luôn chào các sinh viên. Sử dụng các đoạn thông tin được cung cấp phía dưới đây để trả lời câu hỏi từ người dùng.

Nếu người dùng chào bạn, hãy phản hồi: "Chào bạn, đây là trợ lý ảo của trường Đại học Giao thông vận tải. Rất vui được gặp bạn 😊".

Nếu người dùng cảm ơn hoặc bày tỏ cảm xúc tích cực, hãy cảm ơn người dùng. Nếu người dùng phẫn nộ, không hài lòng hoặc biểu đạt cảm xúc tiêu cực, hãy xin lỗi người dùng và xin người dùng góp ý để hoàn thiện hệ thống.

Nếu bạn không thể tìm thấy đáp án từ đoạn thông tin, hãy phản hồi: "Rất Xin lỗi, tôi không thể tìm thấy thông tin phù hợp để trả lời câu hỏi của bạn. Xin vui lòng liên hệ với văn phòng khoa hoặc văn phòng nhà trường để được giải quyết". Bạn không cần phải cố giải đáp hết các câu hỏi của người dùng.

Nếu câu hỏi của người dùng không liên quan đến trường Đại học Giao thông vận tải, không thuộc một trong các chủ đề là học tập, ký túc xá, học phí và bảo hiểm y tế, cơ sở vật chất và hạ tầng, chất lượng giảng dạy, hoạt động ngoại khóa, nghiên cứu khoa học và hội thảo, thư viện, an ninh, học bổng và chính sách, hãy phản hồi: "Xin lỗi bạn, dường như bạn đang hỏi một chủ đề nằm ngoài phạm vi nhà trường. Tôi không thể giải quyết được vấn đề này cho bạn."

Câu trả lời của bạn phải ngắn gọn và đầy đủ ý nhất có thể

Danh sách thông tin:

{source_knowledge}”

Nội dung của prompt gồm ba thành phần chính:

i. *Phần instruction*: Giới thiệu chức vụ của LLM là một nhân viên hỗ trợ sinh viên của trường Đại học Giao thông vận tải, luôn chào sinh viên trước khi trả lời, chỉ hỗ trợ các câu hỏi có liên quan đến trường, và thái độ của LLM nếu nhận được phản hồi tích cực hay tiêu cực từ sinh viên.

ii. *Phần tri thức đi kèm*: Được đánh dấu dưới dạng ô trống ({source_knowledge}) và sẽ được thay thế trong quá trình tích hợp vào LLM.

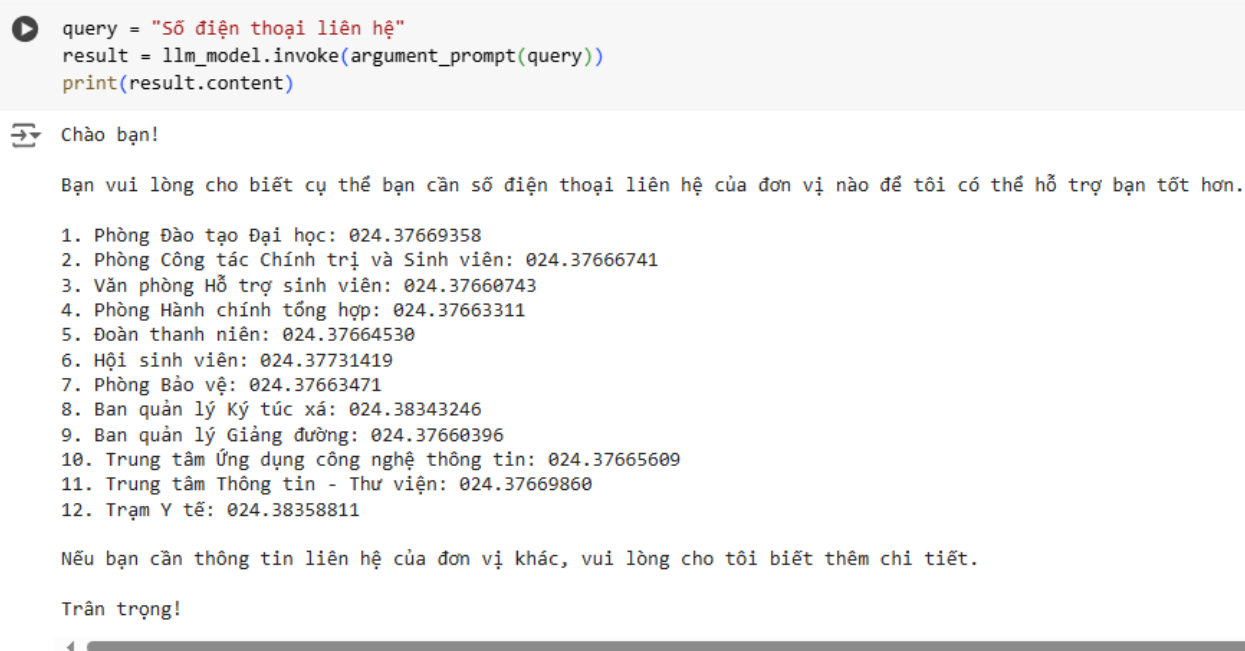
iii. *Phần câu hỏi*: Đã đính kèm trong lịch sử chat của người dung nên không cần cho vào prompt (tự động được đính kèm).

Prompt được thay thế với nội dung câu hỏi thực tế của sinh viên và tri thức đi kèm, sau đó được truyền vào LLM.

Đối với quá trình generating: Ở đây, tôi lựa chọn mô hình gpt-4o của OpenAI. Đây là mô hình mới nhất của OpenAI, có thông số như sau:

- Cửa sổ ngữ cảnh đầu vào: 128000 tokens.
- Số lượng truy vấn tối đa có thể nhận trong một phút: 500 RPM.
- Số lượng token tối đa có thể nhận trong một phút: 30000 RPM.
- Chi phí trên một token: 5 USD / 1M tokens (đối với đầu vào) và 15 USD / 1M tokens (đối với đầu ra).

Do là một mô hình rất mạnh, thông qua quá trình kiểm thử và so sánh với nội dung trong sổ tay sinh viên, mô hình đã trả lời rất chính xác.

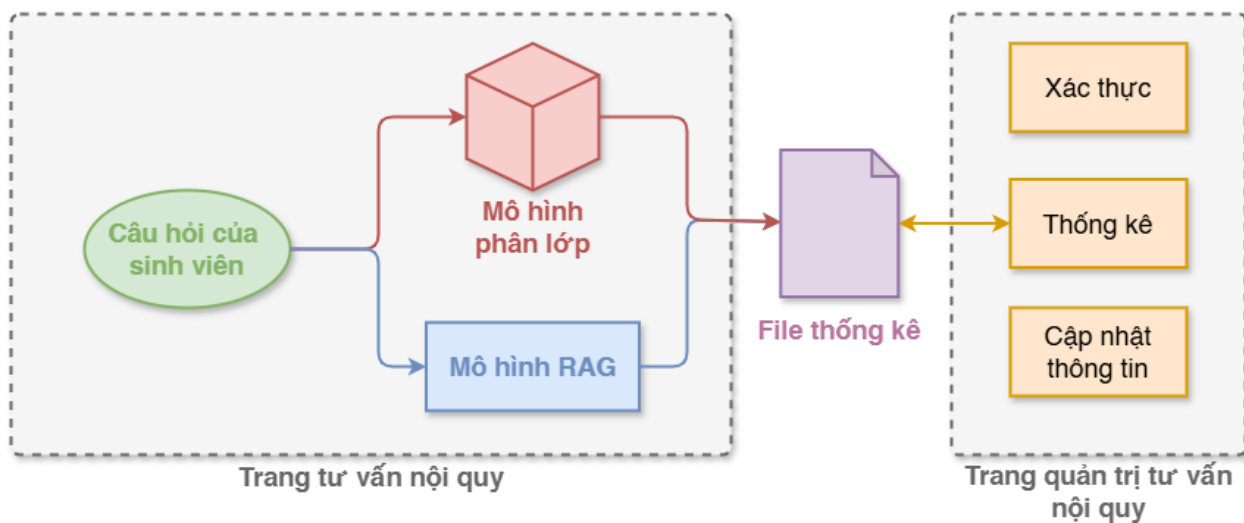


Hình 4.5: Kiểm thử mô hình khi chạy với phương pháp RAG trên số tay sinh viên với câu hỏi "số điện thoại liên hệ".

4.2.2 Phân lớp câu hỏi của người dùng

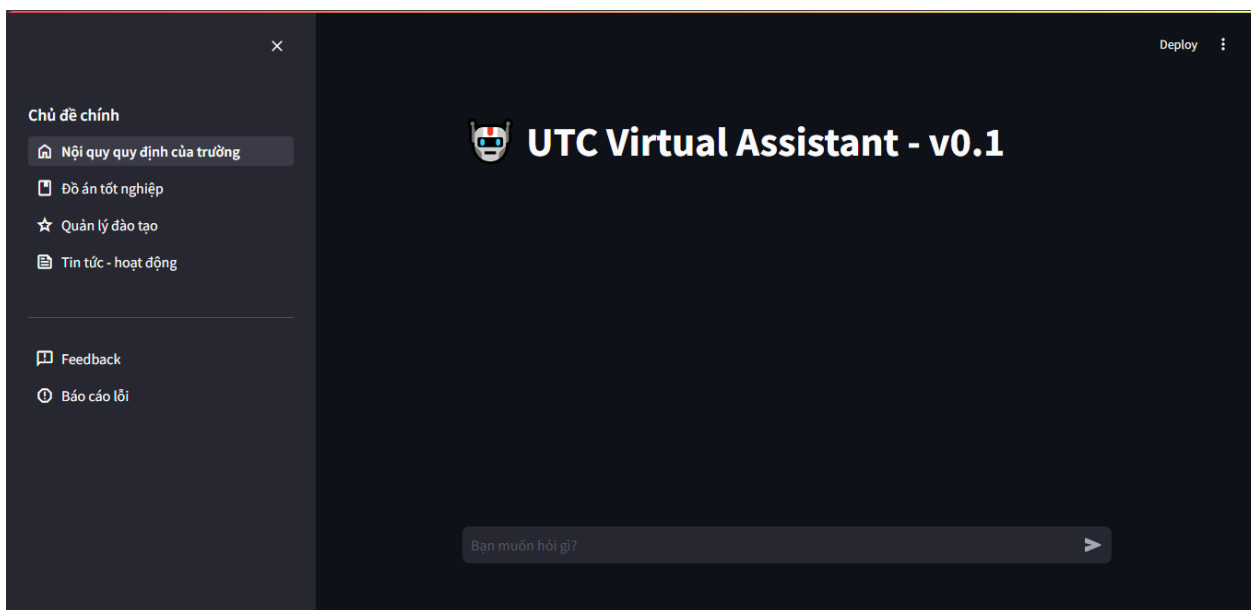
Chủ đề của các câu hỏi của sinh viên cũng là một đề tài cần chú ý. Thông qua chủ đề của các câu hỏi, các thầy cô giáo trong trường có thể nắm bắt được những chủ đề mà sinh viên đang quan tâm. Từ các dữ liệu đó, các thầy cô sẽ có những biện pháp điều chỉnh hoặc đưa ra thêm các giải pháp để giúp đỡ sinh viên. Tuy nhiên, việc phân loại các chủ đề của sinh viên hoàn toàn không thể làm bằng tay, do số lượng câu hỏi của sinh viên lớn, đặt ra nhu cầu hệ thống của tôi phải tự động đánh nhãn được các chủ đề cho từng câu hỏi của sinh viên.

Để giải quyết vấn đề này, tôi đã sử dụng một mô hình phân lớp chủ đề của câu hỏi của sinh viên do tác giả là TS. Lương Thái Lê và Nguyễn Quang Duy thực hiện [17]. Mô hình là sự kết hợp của mô hình `vinai/phobert-base-v2` và mô hình học sâu `bagging` do hai tác giả huấn luyện, được huấn luyện với tập dữ liệu gồm hơn 10000 bình luận của sinh viên trường Đại học Giao thông vận tải, được đánh nhãn gồm 11 lớp: Chủ đề khác, học tập, ký túc xá, học phí và bảo hiểm y tế, cơ sở vật chất và hạ tầng, chất lượng giảng dạy, hoạt động ngoại khóa, nghiên cứu khoa học và hội thảo, thư viện, an ninh, học bổng và chính sách. Kết quả mô hình đạt số điểm F1 trung bình là 0.852 trên tổng số 12 lớp. Do đó, tôi quyết định lựa chọn mô hình này làm mô hình phân lớp cho các câu hỏi của người dùng.



Hình 4.6: Sơ đồ cấu trúc hệ thống tư vấn sinh viên đã bổ sung thêm mô hình phân lớp.

4.2.3 Giao diện trang web tư vấn viên nội quy



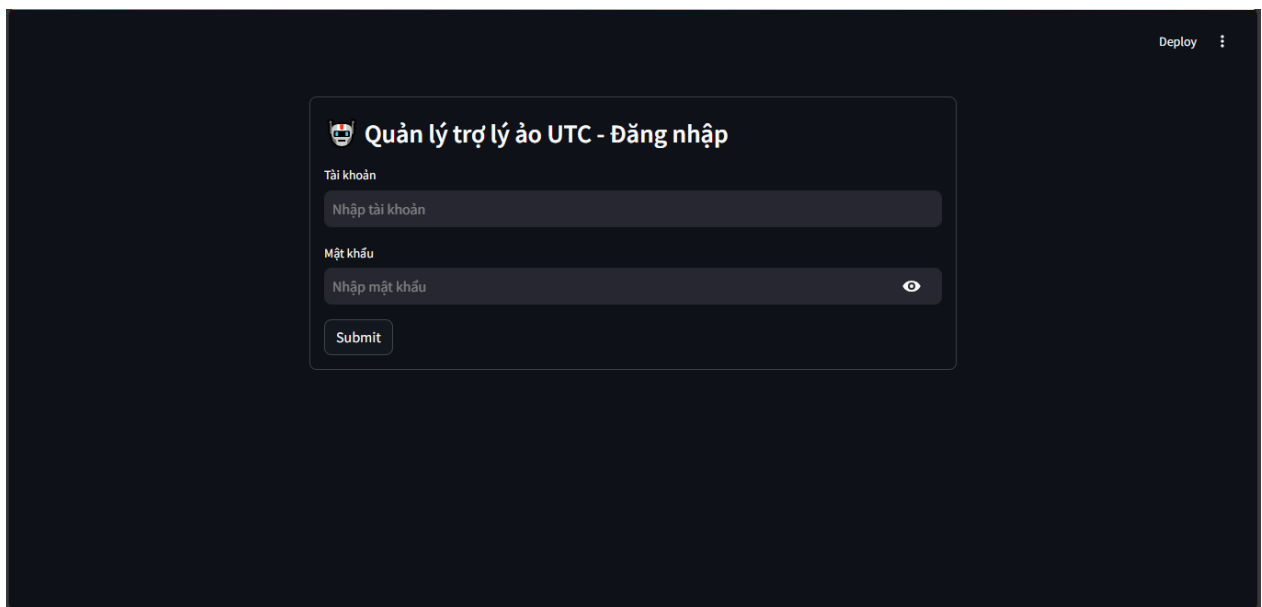
Hình 4.7: Giao diện trang web tư vấn viên nội quy



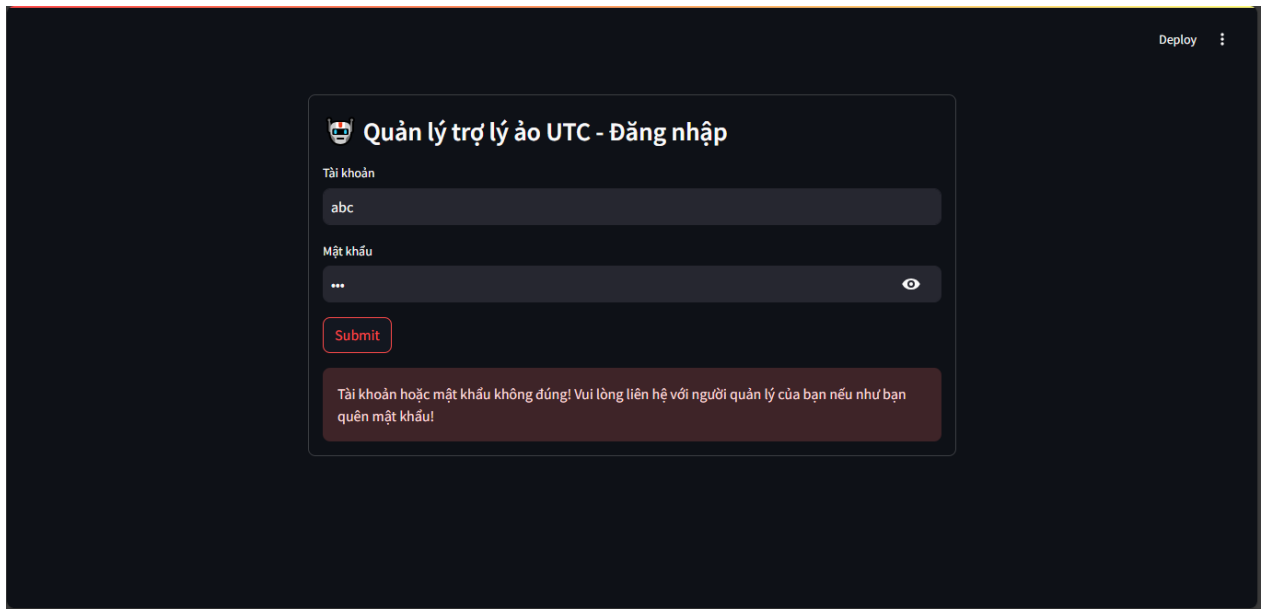
Hình 4.8: Giao diện trang web tư vấn viên nội quy khi sinh viên đặt câu hỏi

4.3 Xây dựng trang web quản trị tư vấn nội quy

4.3.1 Giao diện chức năng xác thực



Hình 4.9: Giao diện trang đăng nhập vào tài khoản quản trị của trang web



Hình 4.10: Giao diện trang đăng nhập báo lỗi khi người dùng nhập không đúng tài khoản

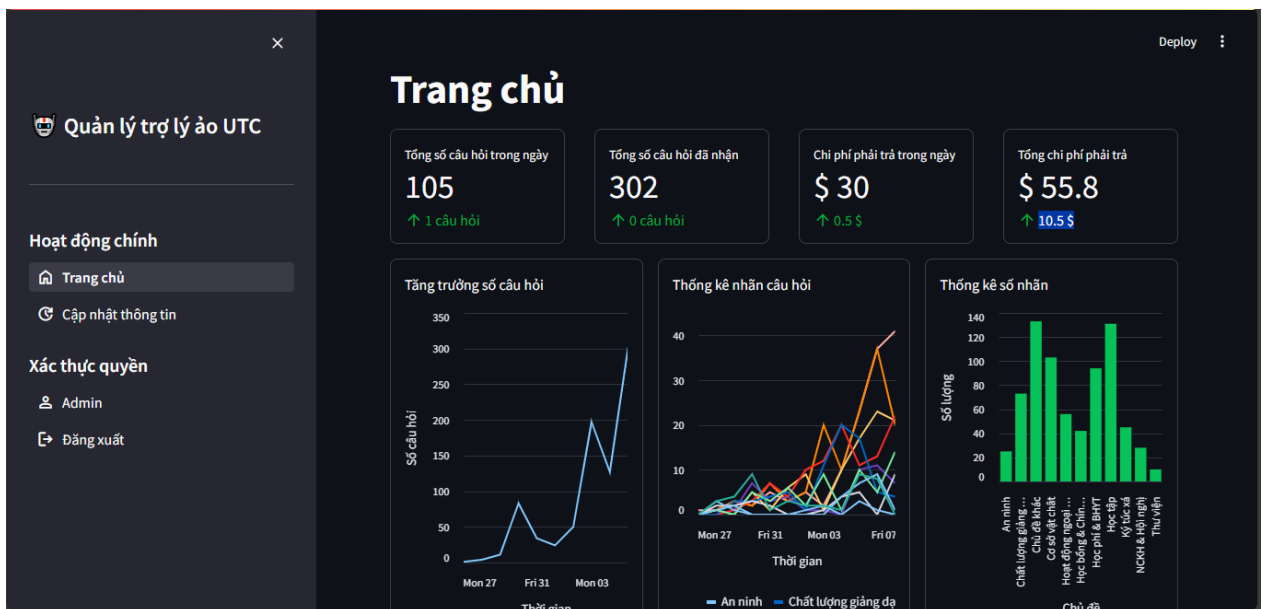


Hình 4.11: Giao diện chức năng đổi mật khẩu của tài khoản quản trị



Hình 4.12: Giao diện chức năng tra cứu biến động chi phí của tài khoản OpenAI

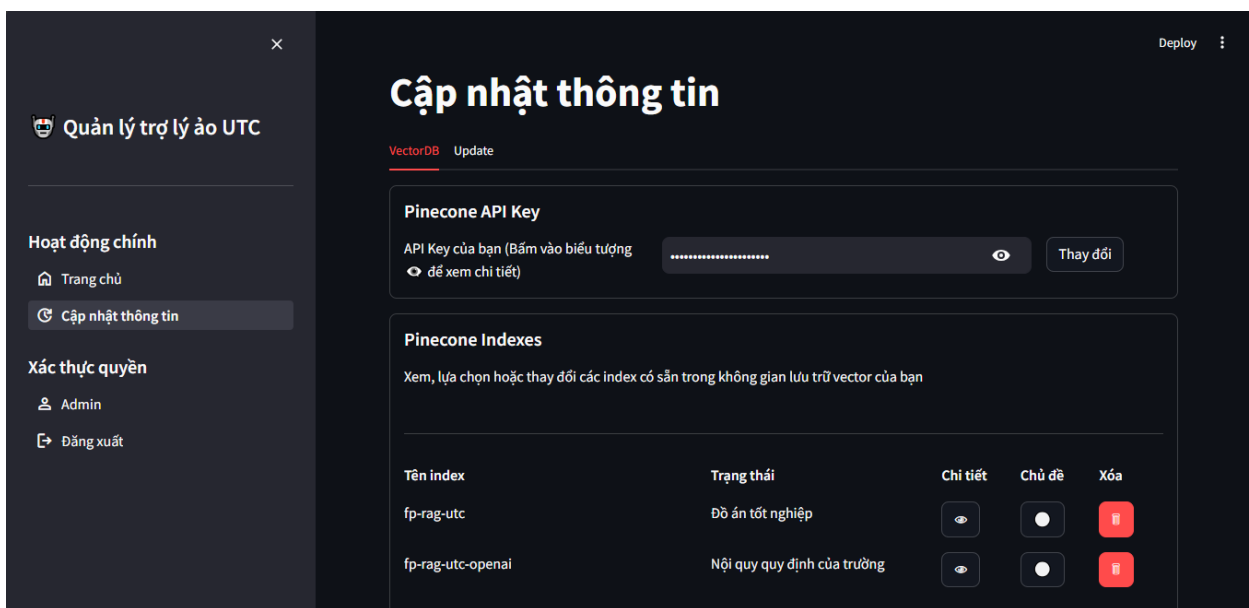
4.3.2 Giao diện chức năng thống kê



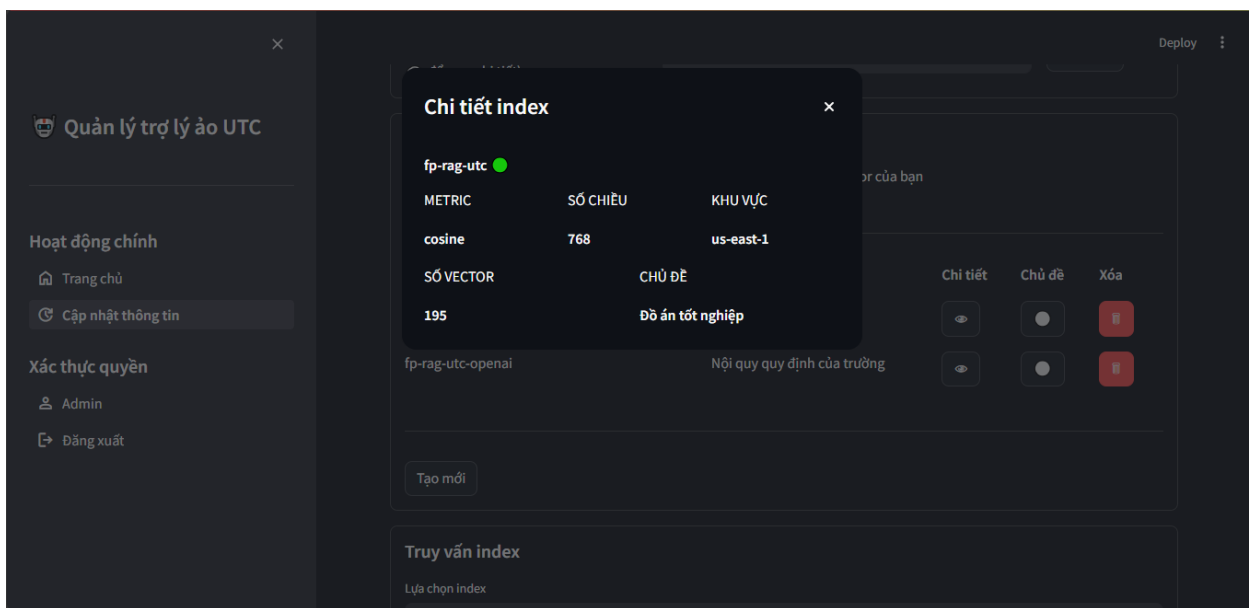
Hình 4.13: Giao diện chức năng thống kê của trang web

4.3.3 Giao diện chức năng cập nhật dữ liệu

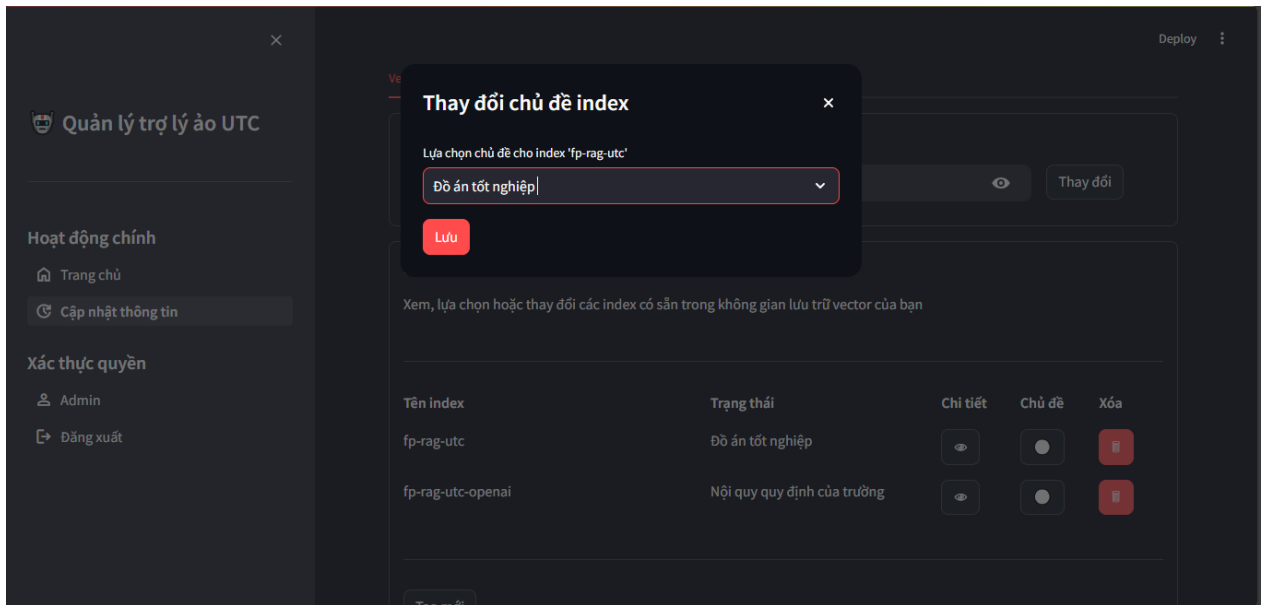
4.3.3.1 Cập nhật dữ liệu về CSDL Vector store



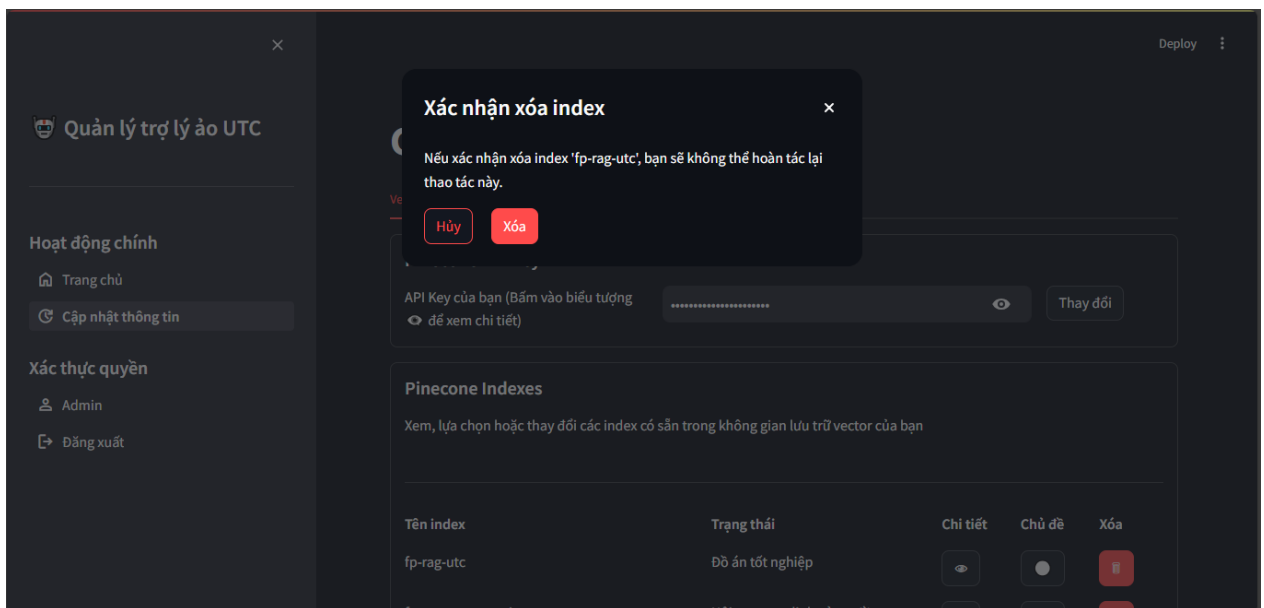
Hình 4.14: Giao diện của chức năng cập nhật thông tin về CSDL vector store



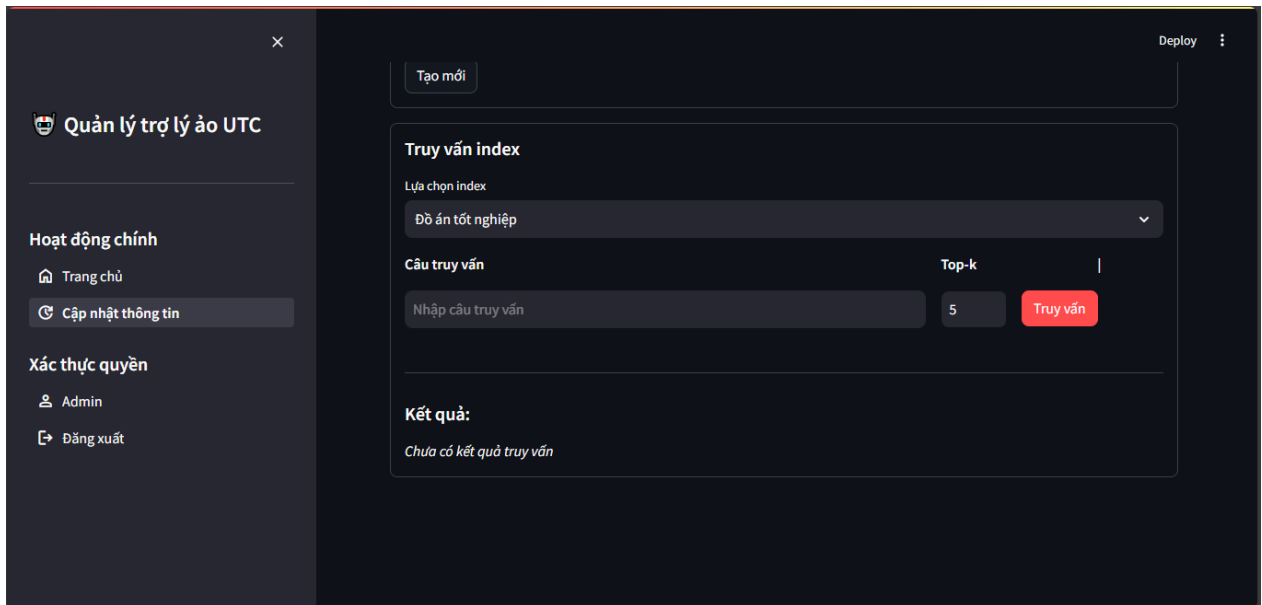
Hình 4.15: Giao diện xem thông tin của index



Hình 4.16: Giao diện thay đổi chủ đề cho index



Hình 4.17: Giao diện xác nhận xóa index



Hình 4.18: Giao diện truy vấn index

4.3.3.2 Cập nhật bổ sung dữ liệu



Hình 4.19: Giao diện cập nhật bổ sung dữ liệu

KẾT LUẬN VÀ KIẾN NGHỊ

Thông qua đề tài đồ án tốt nghiệp, tôi đã xây dựng thành công một hệ thống tư vấn nội quy sinh viên của trường Đại học Giao thông vận tải ứng dụng phương pháp RAG và sức mạnh của LLM. Hệ thống đã có thể hoạt động tốt và sinh câu trả lời chính xác, tự nhiên, giúp cho việc tra cứu các thông tin liên quan đến trường của sinh viên dễ dàng hơn và giúp việc hỗ trợ sinh viên nhanh chóng hơn. Mô hình GPT-4o quả thực rất hiệu quả trong các tác vụ hỏi đáp và RAG. Ngoài ra, việc lưu trữ vector bằng Pinecone cũng rất thuận tiện, dễ truy vấn, dễ thao tác và miễn phí. Sử dụng Streamlit để xây dựng giao diện cho trang web giúp tăng tốc độ thiết kế trang web lên rất cao, xây dựng trang web đơn giản đáp ứng đủ các chức năng cần thiết một cách nhanh chóng

Qua đề tài này, tôi xin rút ra một vài kết luận như sau:

1. Tính mới của đồ án: Đồ án sử dụng phương pháp tương đối mới hiện nay là RAG, kết hợp với mô hình ngôn ngữ lớn rất mạnh của OpenAI là GPT-4o, tận dụng sức mạnh của mô hình để sinh câu trả lời cho câu hỏi của sinh viên từ các tài liệu bổ sung cho LLM.

2. Tính thực tiễn của đồ án: Đồ án có tính thực tiễn rất cao, hoàn toàn có thể ứng dụng để xây dựng một hệ thống thật trên thực tế, giúp tăng hiệu quả quản lý và hỗ trợ sinh viên của nhà trường với chi phí tiết kiệm. Việc quản lý hệ thống cũng đơn giản, hoàn toàn có thể thực hiện chỉ với một nhân sự.

3. Tính mở rộng của đồ án: Đồ án hoàn toàn có thể mở rộng, do được tối ưu dưới dạng module, giúp bổ sung thêm chức năng dễ dàng.

Tuy nhiên, đồ án tốt nghiệp của tôi không thể tránh khỏi những thiếu sót, đó là:

1. Số lượng câu hỏi của sinh viên bị giới hạn: Do mô hình GPT-4o giới hạn số lượng truy vấn trong một phút, do đó nếu số lượng câu hỏi của sinh viên tăng vọt thì hoàn toàn có thể khiến mô hình quá tải.

2. Mô hình GPT-4o là mô hình trả phí và chạy trên điện toán đám mây. Mặc dù mức phí rất thấp, tuy nhiên nếu số lượng truy vấn cao, vượt quá giới hạn chi phí tối đa của mô hình thì mô hình sẽ từ chối truy vấn. Do chạy trên đám mây, nếu hệ thống đám mây đó gặp trục trặc thì mô hình cũng sẽ không hoạt động.

3. Hệ thống mới chỉ ứng dụng phương pháp RAG Ngây thơ trên tập dữ liệu còn hạn chế. Nếu số lượng tài liệu lớn, phương pháp này có thể khiến mô hình kém chính xác hơn.

Qua những ưu điểm và hạn chế ở trên, trong tương lai tôi có một số đề xuất để cải thiện hệ thống hơn như sau:

1. Áp dụng phương pháp RAG nâng cao để tăng hiệu quả phản hồi sinh viên từ LLM
2. Sử dụng một mô hình không trả phí, có thể chạy trên máy nội bộ và ưu tiên các mô hình đặc trưng cho tiếng Việt.
3. Nâng cao số lần truy vấn của sinh viên hơn, có thể cho phép truy vấn các thông tin như bảng điểm, học phí, hoặc truy vấn bằng hình ảnh.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] "Introduction to Large Language Models | Machine Learning | Google for Developers," 08 08 2023. [Online]. Available: https://developers.google.com/machine-learning/resources/intro-llms#what_is_a_language_model.
- [2] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, "Language Models are Unsupervised Multitask Learners," 2019.
- [3] "What are Large Language Models? | NVIDIA," [Online]. Available: <https://www.nvidia.com/en-us/glossary/large-language-models/>.
- [4] Vaswani, Ashish and Shazeer, Noam and Parmar, Niki and Uszkoreit, Jakob and Jones, Llion and Gomez, Aidan N and Kaiser, Łukasz and Polosukhin, Illia, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [5] "Prompt Engineering Guide | Prompt Engineering Guide," [Online]. Available: <https://www.promptingguide.ai/>.
- [6] "Models | OpenAI API," [Online]. Available: <https://platform.openai.com/docs/models/gpt-3-5-turbo>.
- [7] Sebastian Riedel, Douwe Kiela, Patrick Lewis, Aleksandra Piktus, "Retrieval Augmented Generation: Streamlining the creation of intelligent natural language processing models," 28 09 2020. [Online]. Available: <https://ai.meta.com/blog/retrieval-augmented-generation-streamlining-the-creation-of-intelligent-natural-language-processing-models/>.
- [8] Lewis, Patrick and Perez, Ethan and Piktus, Aleksandra and Petroni, Fabio and Karpukhin, Vladimir and Goyal, Naman and Kottler, Heinrich and Lewis, Mike and Yih, Wen-tau and Rocktaschel, Tim and others, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459--9474, 2020.

- [9] "Google's Natural Questions," Google, [Online]. Available: <https://ai.google.com/research/NaturalQuestions/>.
- [10] Jonathan Berant, Andrew Chou, Roy Frostig, Percy Liang, "Semantic Parsing on Freebase from Question-Answer Pairs," vol. Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, p. 1533–1544, 2020.
- [11] Mengqiu Wang, Noah A. Smith, Teruko Mitamura, "What is the Jeopardy Model? A Quasi-Synchronous Grammar for QA," Vols. Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL), p. 22–32, 2007.
- [12] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, Haofen Wang, "{Retrieval-Augmented Generation for Large Language Models: A Survey," *ArXiv*, vol. abs/2312.10997, 2023.
- [13] R. Schwaber-Cohen, "Chunking Strategies for LLM Applications," Pinecone Systems, Inc., 30 06 2023. [Online]. Available: <https://www.pinecone.io/learn/chunking-strategies/>.
- [14] Jurafsky, Daniel; H. James, Martin, "Speech and language processing : an introduction to natural language processing, computational linguistics, and speech recognition," 2000. [Online]. Available: <https://web.stanford.edu/~jurafsky/slp3/>.
- [15] "Quickstart - Pinecone Docs," [Online]. Available: <https://docs.pinecone.io/guides/get-started/quickstart>.
- [16] Luong Thai Le, Nguyen Quang Duy, "Ứng dụng học máy để hiểu các thách thức của sinh viên đại học từ phương tiện truyền thông trực tuyến," vol. Giảng dạy và nghiên cứu khoa học cơ bản, 2024.
- [17] "Pricing | OpenAI," [Online]. Available: <https://openai.com/api/pricing/>.